Intelligent Serverless Distributed Tracing and Bottleneck Diagnosis for Microservice Applications

Project Workbook

By

Anukrithi Myadala
Larry Nguyen
Matthew Tang

October 29, 2026

Advisor: Bhargavi Trivedi, Ph.D.

Chapter 1. Literature Search, State of the Art

Contributed by: Anukrithi Myadala

Literature Search

Our project brings together three areas: distributed tracing, trace sampling, and automated bottleneck diagnosis. At the start of our research, we read into Canopy [1] to see one of the first industry tools aimed to improve trace readability while also taking note of its limitations. Then, we read work that addressed the parts Canopy does not automate. We also read the benchmark and dataset papers that let us evaluate on real production data, alongside a small smoke-test set from the Astronomy Shop's built-in failure flags.

Tracing and trace analysis systems

Dapper [2] introduced the span and trace model used by most distributed tracing systems today. Our base paper, Canopy [1], shows how a production system can collect events from billions of requests, join them into trace trees, extract features, and make the data searchable. Canopy still leaves the final diagnosis to an engineer, who writes queries and reads traces. We want to automate that step. We also use Canopy's idea of closing a trace after a quiet period (its tailer waits for a gap in events before declaring a trace complete), with a shorter window to keep results fresh.

CRISP [3] computes critical paths in large microservice graphs at Uber, but its analysis is batch-based. We plan to compute similar information as traces arrive. Hindsight [4] supports making a retention decision after a trace is complete, which is the reason we use tail-based sampling instead of deciding at the start of a request. LatenSeer [5] is also relevant because it builds causal what-if models from real traces and evaluates them without faults injected by the researchers. Seer [6] is an earlier system that predicts QoS violations in microservices from tracing data; it needs a large training set and cluster access, which we do not have, but it confirms that trace patterns carry enough signal to locate problem services.

Bottleneck detection and root cause analysis

MicroRCA [7] uses an anomaly-weighted service graph and personalized PageRank to locate performance problems. It is our main published comparison method. FIRM [8] uses machine learning and resource metrics to find services that violate SLOs. We cannot use its full setup because our datasets do not include CPU or memory data, but we adapt its ideas of relative importance and congestion to trace timings. BARO [9] is another possible comparison method that uses Bayesian change-point detection. DiagFusion [10] combines traces, logs, and metrics; we treat that as future work because this project is trace-only. Zhou et al. [11] surveyed how engineers debug microservice faults in industry and found that most debugging is still manual trace and log inspection, which matches the gap we are targeting.

Sampling, efficiency, and evaluation

TraceMesh [12] studies streaming trace sampling at scale, and Tracezip [13] reduces storage through compression. Both address the cost of storing telemetry, but neither performs the diagnosis we want. RCAEval [14] gives us a public benchmark with more than 700 labeled failure cases and standard metrics: accuracy@1, accuracy@3, and mean reciprocal rank (MRR). It is our main labeled evaluation; faults we inject into a sample application are only a smoke test. Our other main dataset is the Alibaba cluster-trace-microservices collection [15], which contains production traces and has format details documented by Luo et al. [16]. DeathStarBench [17] is the standard open microservice benchmark suite; we looked at it as a testbed but chose the OpenTelemetry Astronomy Shop for our fault-injection smoke test, since it stays in our e-commerce domain and ships with a load generator and built-in failure flags.

The gap we found is fairly specific. Existing systems help collect traces, compute critical paths, or reduce storage, but engineers still perform much of the diagnosis by hand. Root-cause methods often assume that telemetry is already available and commonly evaluate with injected failures. We did not find a system that combines a serverless, scale-to-zero pipeline with tail-aware bottleneck ranking and evaluation on real production traces. That is the combination we are building.

State-of-the-Art Summary

Instrumentation. OpenTelemetry [18] has become the main vendor-neutral way to instrument applications. Its OTLP protocol carries spans as protobuf or JSON over HTTP or gRPC, and most language SDKs default to protobuf. The OpenTelemetry Collector adds processors such as batching and a tail-sampling processor that can hold spans until a trace completes and then decide what to keep. Our pipeline accepts standard OTLP so that any instrumented application can send data to it with one configuration change, and it implements the tail-sampling decision itself so the whole path stays serverless.

Storage and visualization. Jaeger, Zipkin, and Grafana Tempo [18] provide trace storage, search, service maps, and waterfall views. They are the standard open-source tools, but they do not identify the service most likely responsible for a slowdown; an engineer opens traces one at a time. Jaeger is our manual-diagnosis baseline for exactly this reason. All three run as always-on services with a database or object store behind them.

Commercial APM. Datadog, Dynatrace, New Relic, and Honeycomb add anomaly detection, service-level dashboards, and tail-based sampling on top of the same span data. Their techniques are the closest industry match to what we are building, but the platforms are priced per host and per GB of ingested data and run continuously, which is difficult for small teams and student projects. AWS X-Ray is a natural option on AWS, but its default sampling is head-based and its analysis is limited to service maps and basic latency histograms.

Serverless data infrastructure. The building blocks we need have matured recently. Aurora Serverless v2 can scale down and pause to zero compute capacity when idle, so a

relational store no longer has to run all night. SQS and Lambda scale to zero and charge only per request, and Lambda event-source mappings consume a queue in batches with automatic scaling. These are what make a scale-to-zero tracing pipeline practical today rather than a few years ago.

In short, current tools make traces easier to collect and view, and the commercial tools that do more analysis do it on always-on, usage-priced infrastructure. Our work focuses on making the diagnosis more direct and affordable by combining standard OTLP input, tail-aware ranking, and serverless services that cost close to nothing when no traces are flowing.

References

[1] J. Kaldor et al., "Canopy: An end-to-end performance tracing and analysis system," in Proc. 26th ACM Symp. Operating Systems Principles (SOSP), 2017.
This is the base paper for our project. It describes a production tracing pipeline that collects events, stitches traces, extracts features, and lets engineers query the results. We use its trace-completion idea, but our project adds automated bottleneck ranking.

[2] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Inc., Tech. Rep. dapper-2010-1, 2010.
This paper introduced the span and trace concepts that later tracing systems and OpenTelemetry build on.

[3] Z. Zhang et al., "CRISP: Critical path analysis of large-scale microservice architectures," in Proc. USENIX Annu. Tech. Conf. (ATC), 2022.
CRISP computes critical paths over large microservice traces. Our design applies the same general idea while traces are processed through a stream.

[4] L. Zhang et al., "The benefit of hindsight: Tracing edge-cases in distributed systems," in Proc. 20th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2023.
Hindsight argues that trace retention can be decided after the trace has been observed. This supports our tail-based keep/drop decision at trace completion.

[5] Y. Zhang et al., "LatenSeer: Causal modeling of end-to-end latency distributions by harnessing distributed tracing," in Proc. ACM Symp. Cloud Computing (SoCC), 2023.
LatenSeer uses real traces for causal, what-if analysis. It supports our plan to test whether the evidence behind a diagnosis is meaningful, not only whether a service receives a high score.

[6] Y. Gan et al., "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
Seer predicts QoS violations from tracing data using deep learning. It needs large training sets and cluster control, which we do not have, but it shows that trace patterns carry enough signal to locate problem services.

[7] L. Wu et al., "MicroRCA: Root cause localization of performance issues in microservices," in Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS), 2020.
MicroRCA uses personalized PageRank on an anomaly-weighted service graph. It is the main published method we plan to compare with our score.

[8] H. Qiu et al., "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2020.
FIRM combines machine learning with resource measurements. We use two of its ideas as trace-only signals because our datasets do not contain resource metrics.

[9] L. Pham et al., "BARO: Robust root cause analysis for microservices via multivariate Bayesian online change point detection," in Proc. ACM Int. Conf. Foundations of Software Engineering (FSE), 2024.
BARO is a recent statistical root-cause method based on multivariate Bayesian online change-point detection. We may include it as a second published comparison.

[10] S. Zhang et al., "Robust failure diagnosis of microservice system through multimodal data (DiagFusion)," IEEE Trans. Services Computing, vol. 16, no. 6, 2023.
This work shows a direction beyond our current scope: combining traces with logs and metrics. We will consider that extension only if trace-only diagnosis leaves a clear gap.

[11] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," IEEE Trans. Software Engineering, vol. 47, no. 2, pp. 243-260, 2021.
An industrial survey of how microservice faults are actually debugged. It reports that engineers rely heavily on manual trace and log inspection, which is the gap our automated ranking targets.

[12] Z. Chen et al., "TraceMesh: Scalable and streaming sampling for distributed traces," arXiv:2406.06975, 2024.
TraceMesh studies large-scale streaming trace sampling. It addresses telemetry cost but does not rank likely bottlenecks.

[13] Z. Chen et al., "Tracezip: Efficient distributed tracing via trace compression," in Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA), 2025.
Tracezip focuses on trace compression and storage cost. It is complementary to our tail-based sampling design.

[14] L. Pham et al., "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in Companion Proc. ACM Web Conf. (WWW), 2025.
RCAEval provides more than 700 labeled cases and standard ranking metrics. It is our main source of labeled accuracy; faults we inject ourselves are only a smoke test.

[15] Alibaba Group, "Alibaba cluster trace program: cluster-trace-microservices-v2021 and v2022," 2021-2022. [Online]. Available: https://github.com/alibaba/clusterdata
These public Alibaba traces are our primary production dataset (v2021 covers about 20,000 microservices over 12 hours; v2022 extends the collection to 13 days). We have already inspected sample rows and found a large difference between the median call time and the p99, which supports our focus on tail behavior.

[16] S. Luo et al., "Characterizing microservice dependency and performance: Alibaba trace analysis," in Proc. ACM Symp. Cloud Computing (SoCC), 2021.
This paper explains the Alibaba trace format, including duplicate caller/callee records and response-time semantics that our normalizer must handle.

[17] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
DeathStarBench, the standard open microservice benchmark suite. We evaluated it as a possible testbed, but chose the OpenTelemetry Astronomy Shop for our fault-injection smoke test because it is an e-commerce application with built-in failure flags.

[18] OpenTelemetry, Jaeger, Grafana Tempo, AWS X-Ray, and Datadog APM (tail-based sampling), official documentation. [Online]. Available: https://opentelemetry.io, https://www.jaegertracing.io
These industry tools provide context for our design and are the basis of the state-of-the-art summary. Jaeger also serves as our manual-diagnosis baseline.

Chapter 2. Project Justification

Contributed by: Anukrithi Myadala

Microservice applications split one user request across many services. When one part slows down, the effect can be hard to see from averages alone. In our sample of Alibaba production traces, the median call time is about 1 ms while the p99 is 237 ms, a gap of more than 200x. That gap is why we focus on slow-tail behavior, especially for e-commerce requests where latency affects whether a purchase completes. Industry practice already reflects this: service-level objectives are written against p95 and p99 latency, not averages.

The current tools leave a practical gap. Large companies can operate always-on tracing systems, but engineers still spend time opening traces and looking for the cause; an industrial survey found this manual inspection is still the norm [11]. Open-source tools provide storage and visualizations, while commercial APM tools add analysis at a cost that may not fit a small team. Small teams without a dedicated observability engineer are the ones who lose the most time here, and they are the use case we are targeting.

Our project addresses the gap in two ways. First, the system will stitch incoming traces, compute critical paths, and rank services using several timing signals. Each ranking will include evidence so an engineer can check it, and the system will say so when no service clearly explains a slowdown. Second, the pipeline uses Lambda (with Lambda Function URLs as its public endpoints), SQS, S3, and Aurora Serverless so it can scale down to roughly one to two dollars a month when idle. The main question is whether this lower-cost design can still provide useful diagnoses with acceptable freshness. That is not obvious, which is what makes it a research question and not only an engineering task.

We have framed the work as four research questions so that every result maps to a claim. RQ1: can a serverless pipeline analyze traces as they arrive with bounded freshness and near-zero idle cost? RQ2: does tail-aware critical-path ranking identify the service that best explains slow requests better than simple heuristics and a published method (MicroRCA)? RQ3: how much telemetry can be dropped by tail-based sampling before diagnostic quality degrades, measured as a curve rather than a single number? RQ4: what is the cost, freshness, and quality trade-off of the serverless design compared with an always-on deployment of the same pipeline, in measured dollars?

The scope fits the two-semester project. We plan to build the end-to-end system and a working demo in 295A, then complete broader measurements and reporting in 295B. We also changed the evaluation plan after advisor feedback. Our main evaluation uses real Alibaba traces, the labeled RCAEval benchmark, and an explanation-validity check. We will also inject controlled faults into the OpenTelemetry Astronomy Shop as a smoke test, so we can confirm the ranking finds a known root cause before trusting it on unlabeled data. Injected faults are cleaner than real ones, which is why they are not our headline result. This keeps the evaluation tied to public data and makes the limits of our claims clear. It also gives the project a realistic path to a publishable result: trace-driven online evaluation on production data, with labeled accuracy from

a public benchmark and a measured cost comparison, is a combination we did not find in the literature. For Expo, the demo itself is the argument: replayed production traffic, a bottleneck diagnosed live on the dashboard, and an AWS bill on screen that shows what it cost.

Chapter 3. Project Requirements

Contributed by: Anukrithi Myadala

Requirements

Scope, assumptions, and constraints

- In scope for 295A: the full pipeline from OTLP ingestion to a ranked diagnosis on a dashboard (the live OTLP endpoint is tentative for 295A and moves to 295B if it is not ready), the dataset replay and upload paths, the three baselines, and the experiment runner. Out of scope for 295A: suggested fixes, multi-signal fusion with logs and metrics, and adaptive sampling.
- Assumption: evaluation data comes from public sources (Alibaba traces, RCAEval, the OpenTelemetry Demo). Labeled accuracy comes from RCAEval, whose labels were created by the benchmark authors. We also inject faults into the Astronomy Shop through its built-in failure flags as a smoke test where the root cause is known.
- Constraint: everything runs on AWS within a semester budget of $30–60; no component may require a continuously running server or cluster.

User stories (high level)

- US1. As a developer using OpenTelemetry, I can send OTLP spans to the pipeline with one configuration change and view the resulting traces and rankings.
- US2. As an engineer investigating a slowdown, I can see a ranked list of likely bottleneck services with waterfalls and slow-versus-normal evidence.
- US3. As a user with an OTLP dataset, I can upload the file and run it through the same normalization and analysis path.
- US4. As a researcher on the team, I can rerun an experiment with one command and reproduce the numbers in the report.
- US5. As an advisor reviewing the demo, I can select a slow episode from replayed production data and follow the diagnosis on the dashboard.

Essential features

- E1. Ingestion: accept OTLP/HTTP spans in both protobuf and JSON encodings behind an authentication token. Accepted when the OpenTelemetry Python SDK and the OpenTelemetry Demo can export to the endpoint with only a URL and token configured, and unauthenticated requests are rejected.

- E2. Replay: replay a dataset at its original timing or at 2x and 10x speed, handling the Alibaba dataset's duplicate caller/callee records. Accepted when a replayed hour produces one span per call and inter-arrival times match the original timestamps within the chosen speed-up.

- E3. Upload path: presigned upload for OTLP JSON files. A converter script turns Alibaba CSV and RCAEval traces into OTLP JSON (including Alibaba's caller/callee deduplication) before upload. Accepted when a converted file and the equivalent replay produce identical stitched traces.

- E4. Stitching: stitch spans by trace ID, compute self-time and the critical path, close traces after a configurable quiet window, and label each trace complete, incomplete, or timed out. Late spans must trigger a re-stitch without double-counting. Accepted by unit tests on hand-checked traces from the Alibaba sample.

- E5. Sampling: apply tail-based keep/drop sampling after a trace is complete, keeping slow and error traces plus a configurable random fraction of normal traces. Accepted when the kept set contains every trace above the slow threshold and the configured fraction of the rest.

- E6. Ranking: for each endpoint and time window, rank services using normalized signals (added critical-path latency in slow traces, deviation from the service's normal baseline, slow-request frequency, error rate, and the two FIRM-derived signals). Weights are tuned on the first two-thirds of the data by time and frozen before evaluation on the rest. Accepted when the ranking is reproducible from stored features and the tuning split is recorded.

- E7. Storage: analyzed results in Aurora Serverless v2 PostgreSQL; raw spans in S3 with a 30-day lifecycle rule. Accepted when the database pauses when idle and raw objects expire on schedule.

- E8. Dashboard: a serverless web dashboard with trace search, waterfalls, ranked bottlenecks with evidence, and slow-versus-normal comparisons. Accepted when an engineer can go from a slow episode to a ranked list with supporting traces in under five clicks.

- E9. Baselines: run manual Jaeger (on the same OTLP stream), simple heuristics (slowest average service; longest span in slow traces; critical-path-only ranking), and MicroRCA on the same replayed Alibaba data and on RCAEval. Accepted when each baseline produces a ranking for the same episodes our system does.

- E10. Experiment runner: one command that replays data, runs the pipeline, and writes the results used in the report. Accepted when every table and figure in the report can be regenerated from the runner output.

- E11. Fault-injection smoke test: enable the Astronomy Shop's built-in failure flags, export its traces, and run them through the pipeline. Accepted when the injected service ranks in our top 3.

- E12. RCAEval adapter: convert RCAEval cases and score rankings with accuracy@1, accuracy@3, and MRR. Accepted when our method and every baseline produce scores on the same RCAEval cases.

Desired features

- D1. Flame graph view and an alerting check that flags a sharp rise in a service's score.

- D2. Cost view fed by the actual AWS bill so the scale-to-zero claim is supported by measurements.

- D3. Negative result: report that no service clearly explains a slowdown instead of always forcing a top-1 diagnosis.

- D4. Pipeline health panel showing ingest rate, stitching lag, and timed-out trace rate.

Optional / stretch (deferred to 295B unless time allows)

- O1. A suggested-fix layer on top of the diagnosis (our advisor's suggestion).

- O2. Multi-root episodes and adaptive sampling rates, which need more validation than the first release.

- O3. Combining traces with logs and metrics, considered only if trace-only analysis proves insufficient.

Non-functional requirements

- N1. Freshness: at 1x replay, a completed trace appears on the dashboard with p95 staleness below 60 seconds. Measured with timestamps recorded at ingest, stitch, and store.

- N2. Throughput: the selected Alibaba sample replays at 10x with zero dropped spans; scaling Lambda concurrency during bursts is acceptable.

- N3. Cost: about $1–2 per month while idle, $10–30 per month during active experiments, and $30–60 for the semester, with billing alarms at $10, $25, and $50.

- N4. Correctness: at-least-once delivery must not corrupt stitching or double-count results; writes are idempotent and keyed for deduplication. Verified by replaying the same batch twice and checking the stored results are unchanged.

- N5. Reproducibility: infrastructure as code with one deploy and one destroy command; all evaluation numbers regenerate from the experiment runner.
- N6. Security: no unauthenticated public ingestion endpoint; secrets kept out of the repository. The public research datasets contain no personal data, so no PII handling is required.
- N7. Failure handling: incomplete and timed-out traces are flagged and excluded from rankings; their rate is shown on the health panel so a broken pipeline cannot look like a quiet one.

Chapter 4. Dependencies and Deliverables

Contributed by: Matthew Tang

Dependencies

- AWS cloud services
    - Architecture uses AWS services so loss of AWS access or restrictions would require significant architectural changes
- Usable distributed trace data
    - The project depends on either generating suitable OpenTelemetry traces from an instrumented microservice application or adapting an existing dataset such as Alibaba.
- Trace reconstruction and critical-path feasibility:
    - Our proposed diagnosis algorithm depends on correctly reconstructing traces and deriving certain information. If our dataset does not contain the required information, the dataset or analysis methodology would need to change.
- Ground truth for diagnosis evaluation:
    - Evaluating bottlenecks requires knowing which service is responsible for each test fault. The project depends on the ability to create reproducible fault-injection scenarios or obtain appropriately labeled trace data.

Deliverables

Web-based Bottleneck Diagnosis Application: A functional web application that allows users to upload distributed trace datasets and view analysis results through a dashboard.

AWS Serverless Trace Processing Pipeline: An implemented cloud pipeline for ingesting raw trace data (uploaded datasets and live OTLP spans), normalizing spans, reconstructing traces, and passing reconstructed traces to the diagnosis engine using Amazon S3, SQS, AWS Lambda (with Lambda Function URLs as its public endpoints), and Aurora Serverless v2.

Trace Reconstruction Engine: A component that reconstructs end-to-end requests from individual spans and derives service dependencies, span/self-time information, and critical-path information required for diagnosis.

Bottleneck Diagnosis and Ranking Engine: A trace-based diagnosis component that ranks suspected bottleneck services using a proposed multi-signal bottleneck score incorporating baseline deviation, critical-path contribution, normalized self-time, and error/timeout evidence.

Diagnosis Dashboard: A user-facing interface that presents reconstructed trace information, ranked bottleneck candidates, and the supporting evidence behind each ranking. A suggested-fix layer on top of the diagnosis is planned for CMPE 295B.

Evaluation Dataset and Experimental Results: Reproducible results on real Alibaba production traces (with an explanation-validity check) and on the labeled RCAEval benchmark, plus fault-injected Astronomy Shop traces with known bottlenecks as a smoke test. Our method is compared against manual diagnosis in Jaeger, simple heuristics (slowest average service, longest span, critical-path-only ranking), and MicroRCA using accuracy@1, accuracy@3, and mean reciprocal rank (MRR), along with freshness, throughput, and cost measurements.

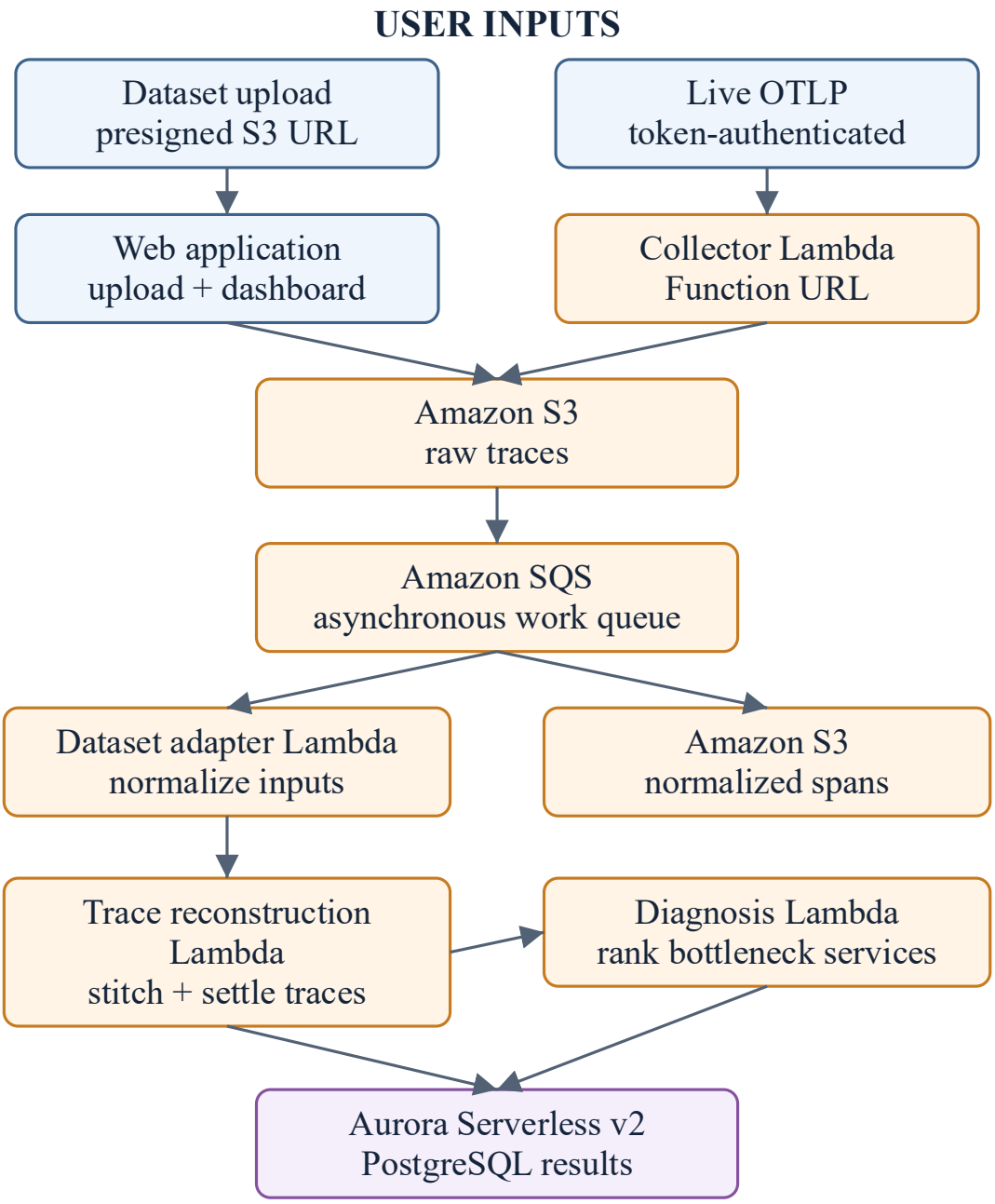
Source Code Repository: A GitHub repository with the pipeline, dashboard, infrastructure as code (one deploy and one destroy command), and the experiment runner, documented so the results can be reproduced.

Project Reports and Expo Demonstration: The CMPE 295A/B workbooks and final project report, and a live demo at the Project Expo showing replayed production traffic diagnosed on the dashboard alongside its measured AWS cost.

Research Paper Draft: A paper draft describing the serverless design and the evaluation results, to be submitted if the results support a publishable contribution.

Chapter 5. Project Architecture

Contributed by: Matthew Tang



Raw and normalized traces flow asynchronously; the dashboard reads stored results.

Fig. 1. System architecture.

Our project uses a serverless, event-driven architecture on AWS to process distributed tracing data and identify potential performance bottlenecks in microservice applications. Every component is either pay-per-use or pauses when idle, so the system scales to zero when no traces are flowing.

Fig. 1 shows our project architecture. The flow starts off with the user uploading their preferred dataset to a web application or getting trace data from a live path. Rather than sending a large trace file through the application server, the dataset is uploaded directly to Amazon S3 using a presigned URL. On the live path, a collector Lambda behind a Lambda Function URL checks the request token and writes batches of OTLP spans to the same raw S3 bucket. S3 serves as the initial persistent storage layer for raw trace data.

After the data is uploaded, an event is placed into Amazon SQS. This decouples data ingestion from processing, allowing trace-processing jobs to be handled asynchronously rather than forcing the user to wait for analysis during the upload request. This then gets passed on to a Lambda dataset adapter which normalizes accepted inputs into a common normalized span representation. We do this to allow the later analysis stages to operate on a consistent trace schema regardless of the original dataset format. These then get stored into a separate S3 location.

A second lambda function performs trace reconstruction. Individual spans are grouped using trace and parent-child relationships to reconstruct the end-to-end execution path of each request. Reconstruction will allow the system to derive information needed for analysis, including service dependencies, span durations, self-time, and critical-path information. Because spans from one request can arrive at different times, a trace is treated as complete once no new spans have arrived for a short quiet window. At that point a tail-based sampling decision keeps slow and error traces plus a small random fraction of normal ones, and drops the rest.

The reconstructed traces get passed to another lambda function, which is the diagnosis component. The diagnosis assigns each service a bottleneck score using multiple sources of trace-derived evidence. The goal is to avoid simply identifying the service with the longest span. Everything ends up being stored in Aurora Serverless v2 (PostgreSQL), which pauses when idle and is accessed through the Data API. The FastAPI backend runs in a Lambda with a Function URL, queries the database, and exposes the results to the frontend/dashboard, where users can inspect the traces, bottleneck candidates, and supporting evidence. Suggested fixes (the "recommendation" step in Fig. 1) are planned for CMPE 295B.

Chapter 6. Project Design

Contributed by: Anukrithi Myadala (design overview, data model, ingestion, live path, analysis, dashboard, evaluation harness, trade-offs), Larry Nguyen (reconstruction and storage; bottleneck score), Matthew Tang (cloud deployment)

This chapter turns the architecture in Chapter 5 into a design we have mostly already built. Every diagram below was drawn from the code in the team repository (github.com/LarryTNguyen/MastersProject) as of October 22, 2026, so the design and the implementation match. Where a part is still planned (the diagnosis scoring step and the AWS deployment), the text says so.

Design overview and principles

The system is five Python packages plus one HTML page, connected through files and one database rather than through direct calls. Fig. 2 shows the packages, their owners, and what flows between them. The boundaries were chosen so each member can test their part alone: the ingest package produces canonical JSONL files, the reconstruction package consumes them into PostgreSQL, and the dashboard reads PostgreSQL. A JSON Schema (trace schema v1) is the contract in the middle.

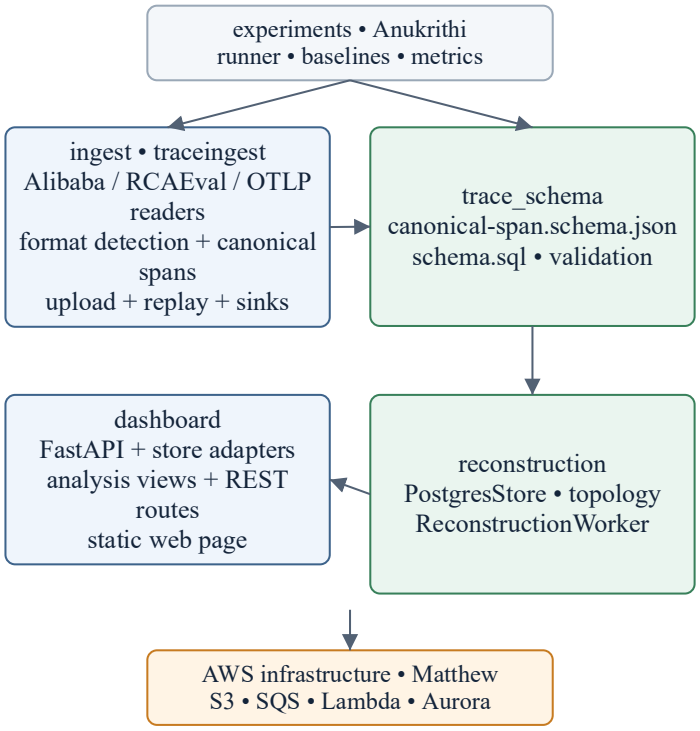


Fig. 2. Package and component map. Ingest and dashboard components are blue; the experiments package is gray (Anukrithi); schema and reconstruction components are green (Larry); AWS infrastructure is orange (Matthew).

Four rules guided the design:

- One span format. Every input (Alibaba CSV, RCAEval traces, OTLP JSON, live OTLP) is converted once, at the edge, into the same record. Everything downstream handles exactly one shape, which is why the dashboard and the reconstruction step did not need per-dataset code.
- Idempotent everywhere. S3 events and SQS deliver at least once. Span identity is (run_id, trace_id, span_id) and the database function ingest_span returns DUPLICATE for a repeat instead of storing it twice, so a retried Lambda cannot double count.
- Evidence with every answer. The API never returns a bare service name: rankings carry a score, a confidence label and the signals behind them, and the compare endpoint returns the slow-versus-normal numbers that justify a ranking. When no service stands out the window carries a note saying so (requirement D3).
- Scale to zero. Every piece is a Lambda, an S3 bucket, a queue, or Aurora Serverless with auto-pause. Nothing in the design needs a process that runs while no traces are flowing; the quiet-window settlement uses a once-a-minute poll instead of a resident worker.

Data model: the canonical span record

Table I lists the record that every adapter emits and that the spans table stores as JSONB. It is defined in trace_schema/canonical-span.schema.json (schema_version 1.0.0) and validated by trace_schema/validate.py; the adapters write it through ingest/traceingest/canonical.py. Timestamps are nanoseconds stored as strings so no precision is lost in JSON. The three quality fields (time_domain, timing_quality, parent_relation) exist because the datasets differ: OTLP spans have real clocks, RCAEval traces have real clocks, and Alibaba rows only have durations and a call tree.

Field	Type	Meaning / allowed values
schema_version	string	"1.0.0"
run_id	uuid	ingestion run (one replay, one upload, or one live session)
identity_kind	enum	OTEL (ids are real OTel hex ids) or DATASET (ids derived from the dataset)
trace_id, span_id, parent_span_id	string	OTEL: 32/16 hex chars; DATASET: blake2b of (trace, rpc id)
parent_relation	enum	ROOT, EXPLICIT, INFERRED (parent derived from rpc id prefix), UNKNOWN
service_namespace, service_name, service_instance_id	string	service identity; instance may be null
operation_name	string	endpoint or RPC name; null when the dataset has none
record_kind	enum	SPAN or CALL_OBSERVATION (Alibaba caller/callee rows)
span_kind	enum	INTERNAL, SERVER, CLIENT, PRODUCER, CONSUMER, UNKNOWN
start_time_ns, end_time_ns, duration_ns	int as string	nanoseconds; start/end may be null for DURATION_ONLY data
time_domain	enum	UNIX, RELATIVE (replay clock), UNKNOWN
timing_quality	enum	MEASURED, DERIVED, DURATION_ONLY, SYNTHETIC, UNKNOWN

Field	Type	Meaning / allowed values
status	object	code UNSET / OK / ERROR / UNKNOWN plus optional message
attributes	object	typed values {type, value}; includes sta.received_us and sta.normalized_us stamps used for freshness
received_at_unix_ns	int as string	when the collector or adapter first saw the record
source	object	format, dataset, raw_reference (S3 key + line), replay_start_ns
quality_flags	array	e.g. negative_rt_dropped, duplicate_rpc_merged

TABLE I

CANONICAL SPAN RECORD (TRACE SCHEMA V1)

Table II shows how each input maps onto that record. The two hard cases are Alibaba and OTLP. Alibaba's call-graph CSV lists every call twice (once from the caller, once from the callee) with response times that can be negative or missing; the reader keeps one record per (traceid, rpcid), keeps the larger of the two response times (the caller side includes the network) and the earliest timestamp, drops negative values with a quality flag, and derives the parent from the rpc id ("0.1.2" is the child of "0.1"). OTLP arrives as resourceSpans → scopeSpans → spans with service.name in resource attributes; the reader flattens that and keeps the ids as they are.

Input	Detected by	Ids	Timing	Service / operation	Special handling
Alibaba call graph CSV	header has traceid, rpcid, um, dm, rt	DATASET; blake2b(trace id, rpcid)	DURATION_ ONLY (rt in ms); start null	dm (callee) / interface	dedupe caller/callee rows (keep larger rt, earliest ts); negative rt → null + flag; rpc id prefix → parent
RCAEval traces (CSV or Parquet)	header has traceID, spanID, serviceName, startTime, duration	DATASET	MEASURED, UNIX (µs → ns)	serviceName / operationNa me	case folders carry inject_time and the true service for scoring
OTLP JSON (file export or collector)	"resourceSpans" in the first 4 KB	OTEL (hex ids kept)	MEASURED, UNIX	resource service.name / span name	status.code mapped UNSET/OK/ERR OR; attributes unwrapped
Canonical JSONL	first line has trace_id and span_id	as stored	as stored	as stored	pass-through, re-validated

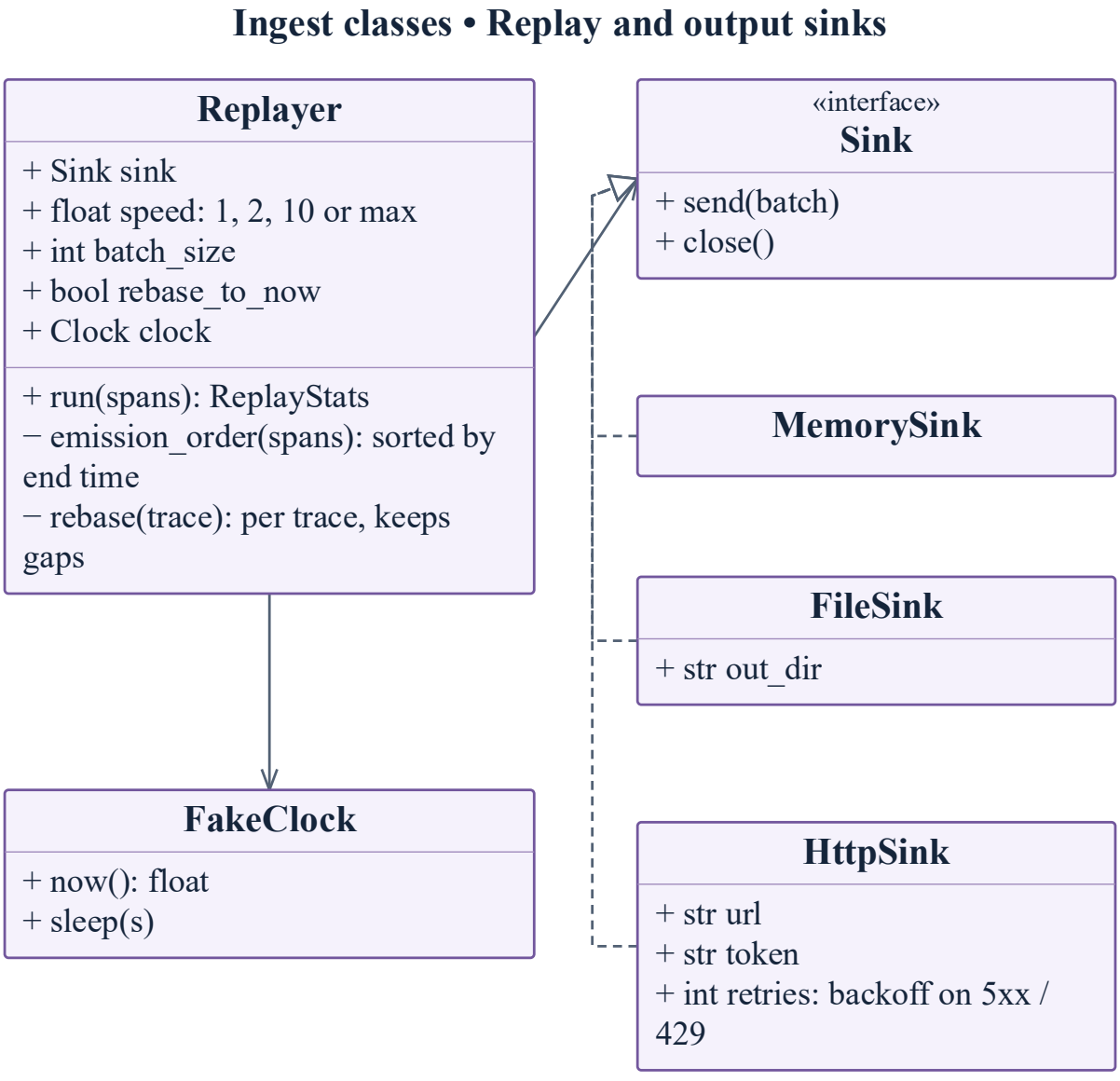
TABLE II

ADAPTER MAPPING FOR THE THREE INPUT FORMATS (PLUS CANONICAL PASS-THROUGH)

Ingestion design: adapters, replayer and upload path

Fig. 3 is the class diagram of the ingest package. The format readers are plain modules that return a list of Span objects; formats.detect_format looks at the first 8 KB so a caller does not need to say what a file is. The Replayer plays spans back as if an application were emitting them: it sorts by end time (a real tracer exports a span when it finishes, not when it starts), batches

them, and sleeps according to the original gaps divided by the speed factor (1x, 2x, 10x or max). With rebase_to_now it shifts every trace so the first span starts now while keeping the gaps inside the trace, which is what makes a 2024 dataset look live on the dashboard. A FakeClock replaces the real one in tests, so the timing logic is tested without waiting.



Replayer uses a Sink and a clock. The three sink classes implement Sink.

Fig. 3(a). Ingest package class diagram: trace replay, clock and output sinks.

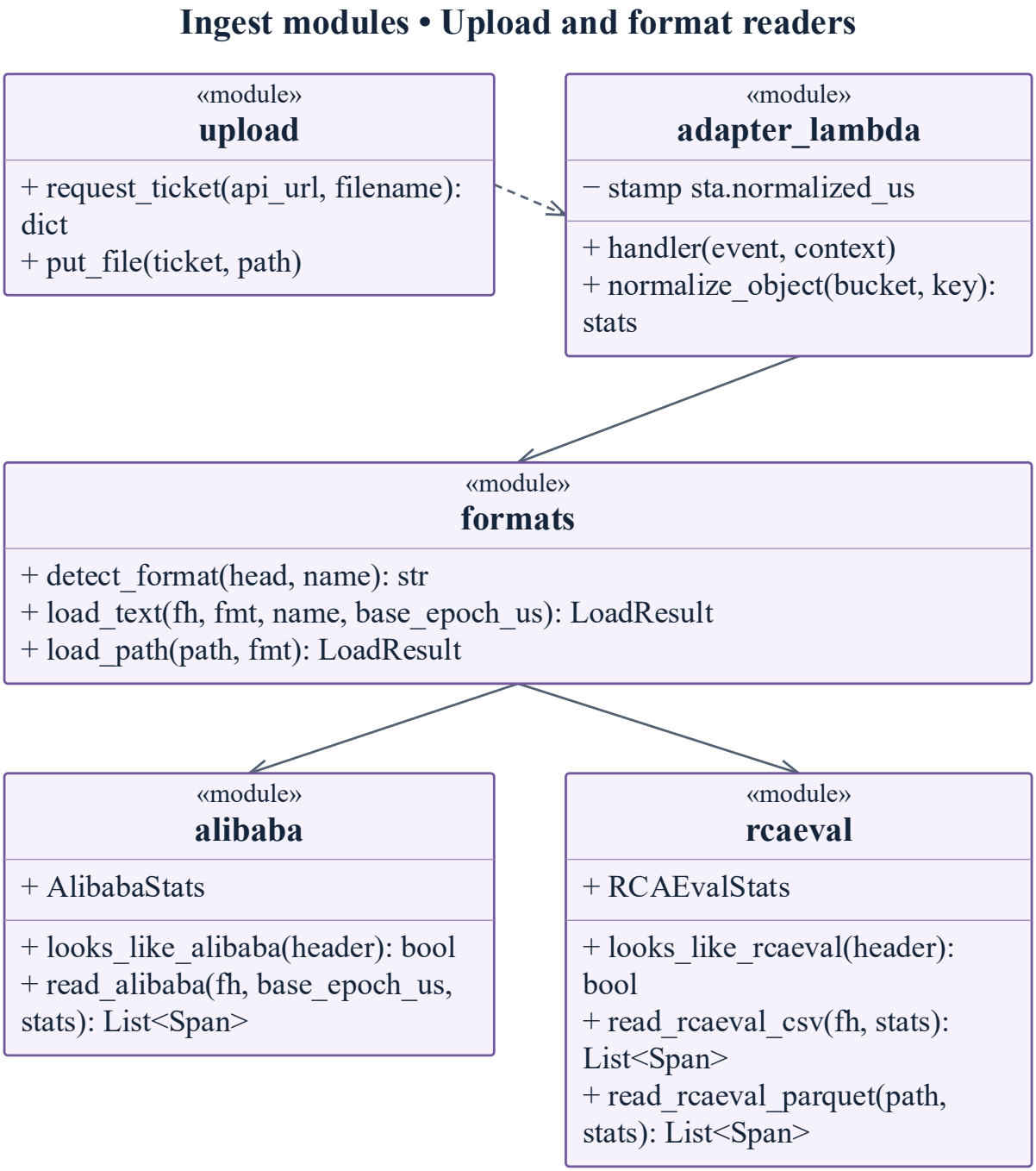


Fig. 3(b). Ingest package class diagram, continued: upload, normalization adapter and dataset readers. Related canonical and span types appear in panel (c).

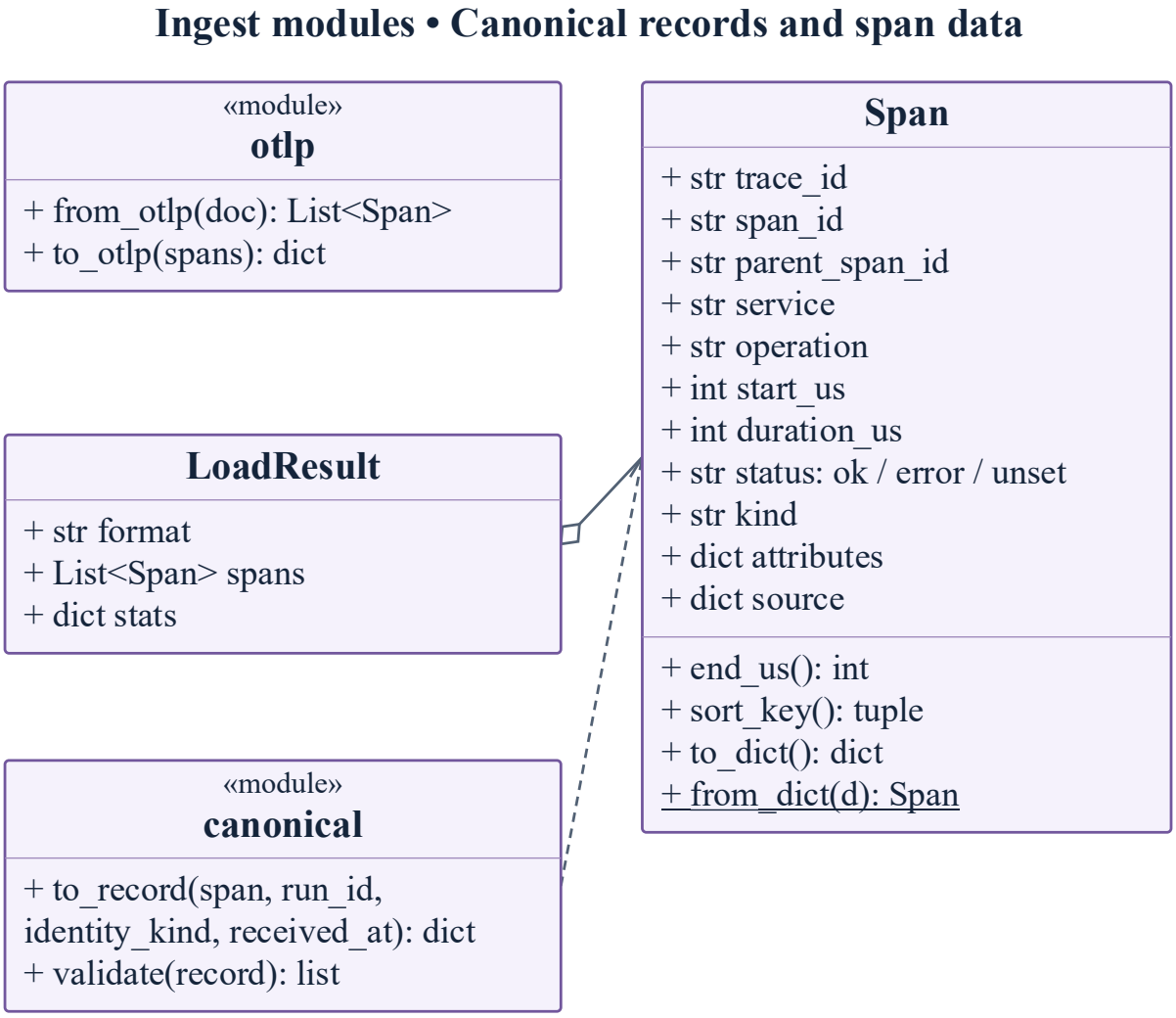


Fig. 3(c). Ingest package class diagram, continued: OTLP conversion, canonical records, LoadResult and Span.

Fig. 4 is the upload path end to end. The dashboard API only hands out a presigned S3 PUT URL (requirement E3); the file itself never passes through a Lambda, so a 300 MB dataset is not limited by the 6 MB Lambda payload. The adapter Lambda is triggered through SQS so that a burst of uploads is queued instead of dropped, and it reports partial-batch failures so one bad file does not block the others. The adapter stamps sta.normalized_us on every record; the dashboard's pipeline view uses the difference between that stamp and the span's end time as the freshness measurement (requirement N1).

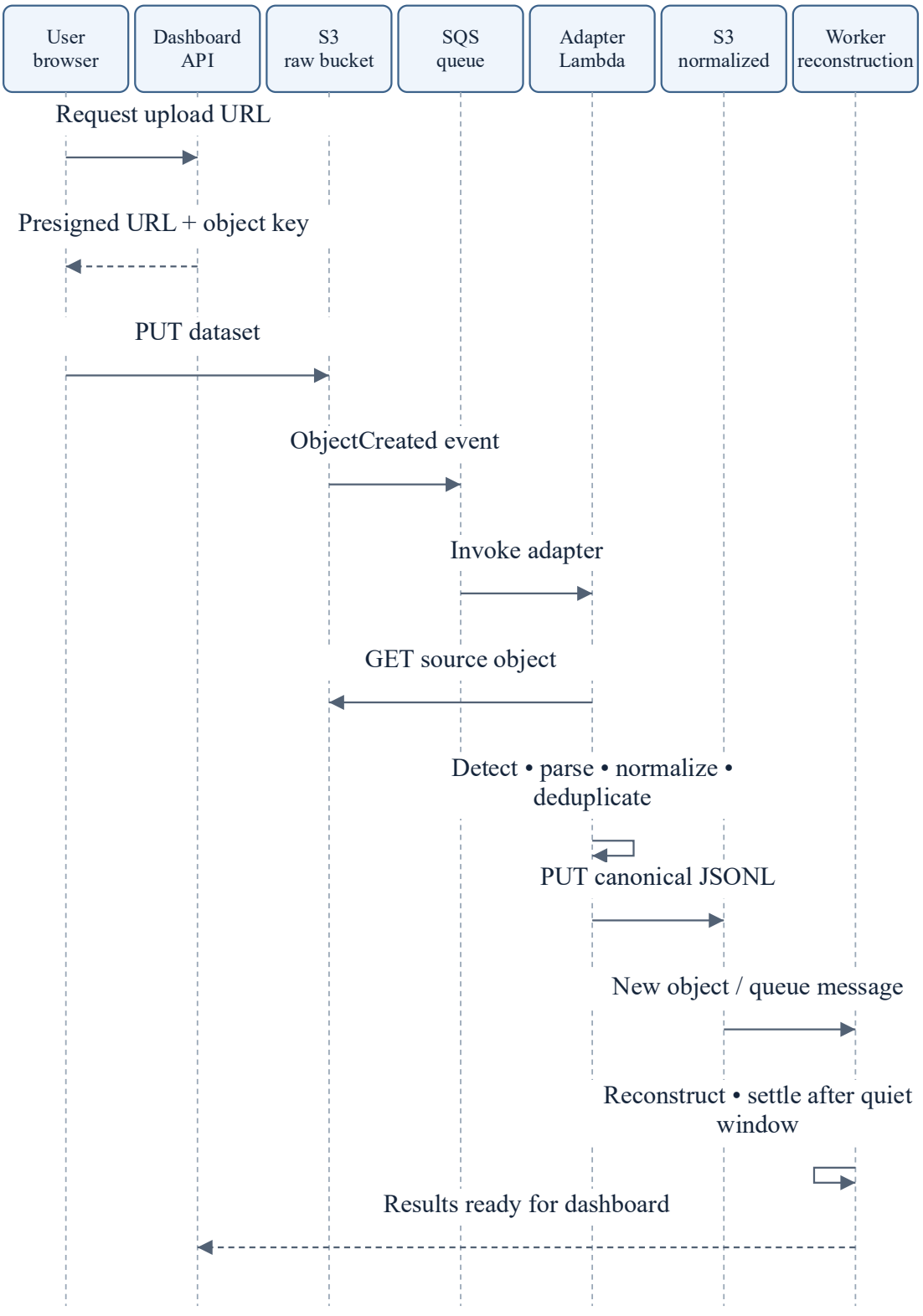


Fig. 4. Sequence diagram, dataset upload path: presigned upload → S3 → SQS → adapter → normalized S3 → reconstruction → PostgreSQL → dashboard.

Sinks are the output side of the replayer: MemorySink for tests, FileSink to write canonical JSONL locally, and HttpSink to POST OTLP JSON to the collector with a bearer token and retries with backoff on 429/5xx. The same replayer therefore feeds both the local pipeline (compose.yaml) and the AWS one.

Live OTLP path

Fig. 5 shows the live path (requirement E1). The collector Lambda does only four things: check the token, check size and encoding, write the batch to the raw bucket with a received_at stamp, and return 200. All parsing happens later in the adapter, so the collector's p99 stays low and an application exporting traces is never slowed down by our analysis. Protobuf and JSON encodings are both accepted because the OpenTelemetry SDKs default to protobuf while file exports and curl tests use JSON.

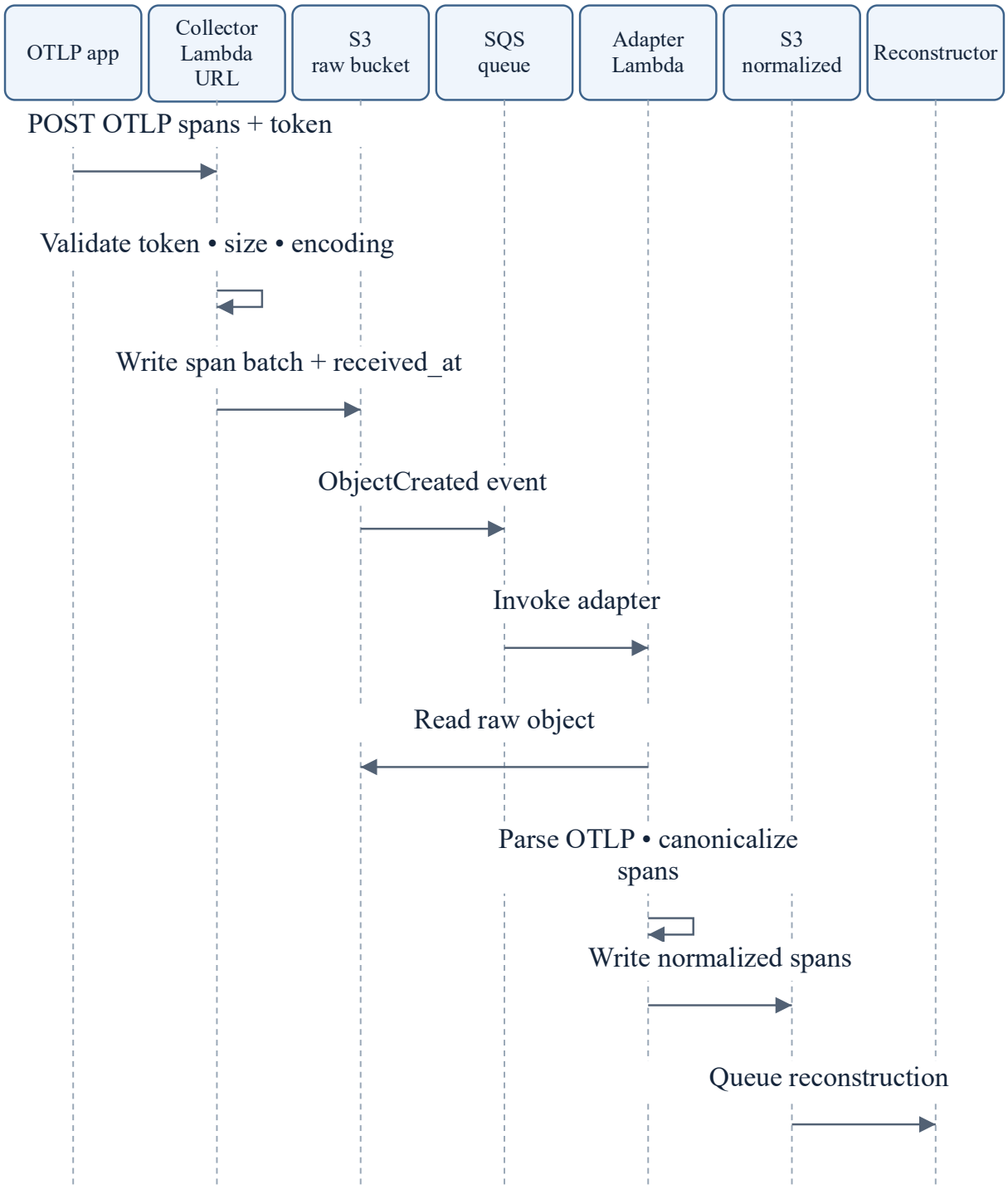
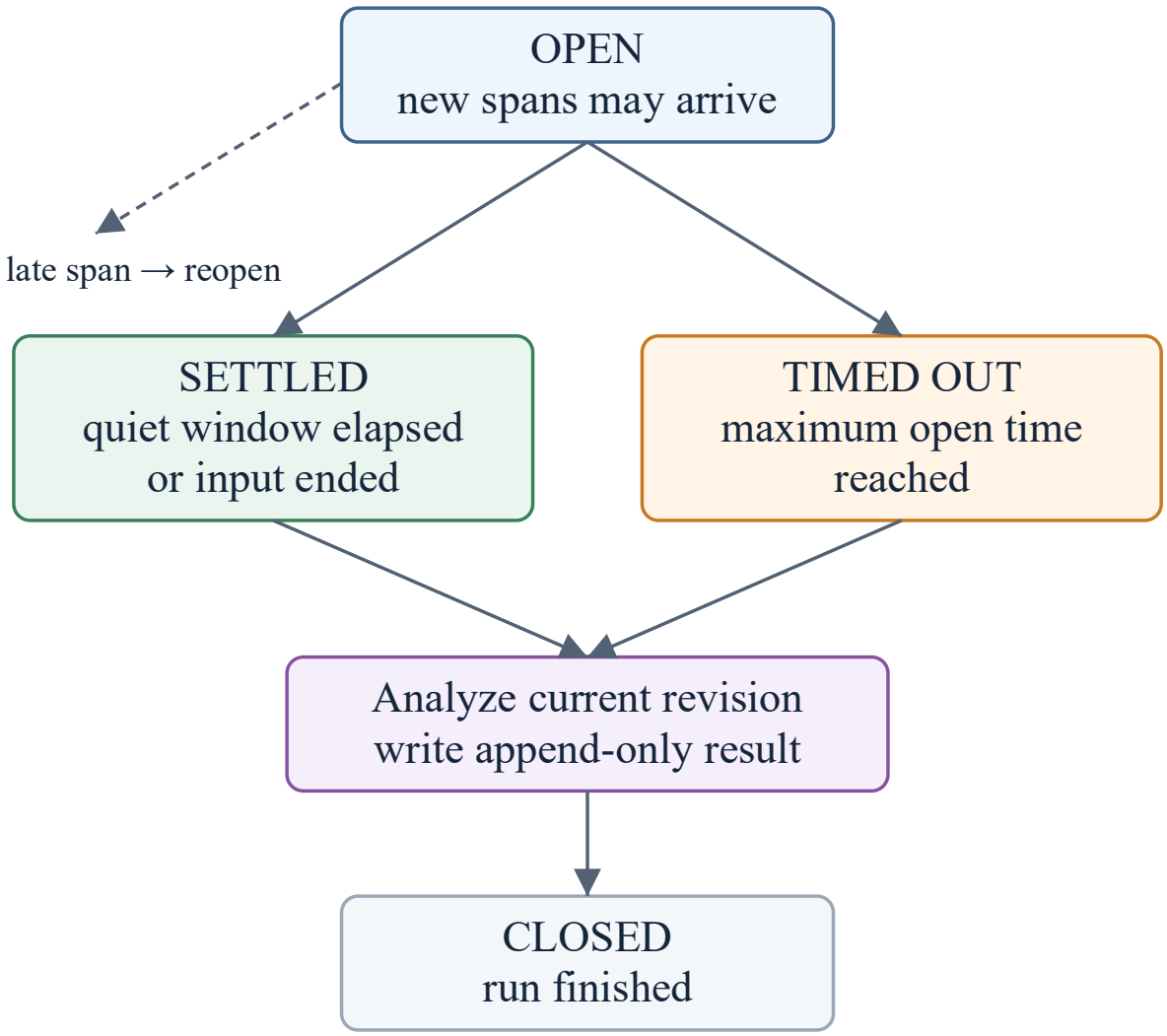


Fig. 5. Sequence diagram, live OTLP path through the collector Lambda.

Trace reconstruction and storage design

Contributed by: Larry Nguyen (drafted from the reconstruction package and schema; to be reviewed by Larry)

The reconstruction package owns the lifecycle of a trace. Spans arrive out of order and over time, so a trace is a revisioned object rather than a row that is written once. Fig. 6 is the state machine. A trace opens when its first span arrives. It settles when no span has arrived for quiet_window_ms (default 5 s), or when the run is explicitly finished (END_OF_INPUT), and it times out at max_open_ms (default 60 s) so a trace with a missing root cannot stay open forever (requirement N7). A late span after settlement reopens the trace and bumps its revision; every analysis row is keyed by the revision it was computed from, so re-analysis replaces instead of adding (requirement E4, no double counting).



Each settled revision is analyzed once; late spans create a new revision.

Fig. 6. Trace lifecycle state diagram (trace_v1.traces.state and settlement_reason).

Fig. 7 is the entity-relationship diagram of the PostgreSQL schema (trace_schema/schema.sql, schema trace_v1). The spans table stores the canonical record as JSONB and exposes the fields

the queries need as generated columns, so the record can gain fields without a migration while service, time and status stay indexed. Every table that holds a result (trace_analyses, service_baselines, service_window_results) is append-only: a trigger rejects UPDATE, and a guard trigger refuses an analysis whose input_revision is not the trace's current settled revision. Two views give the dashboard what it needs without joins in application code: current_trace_analyses (the latest analysis per settled trace) and trace_inventory (per-trace span counts, error counts and observed time range).

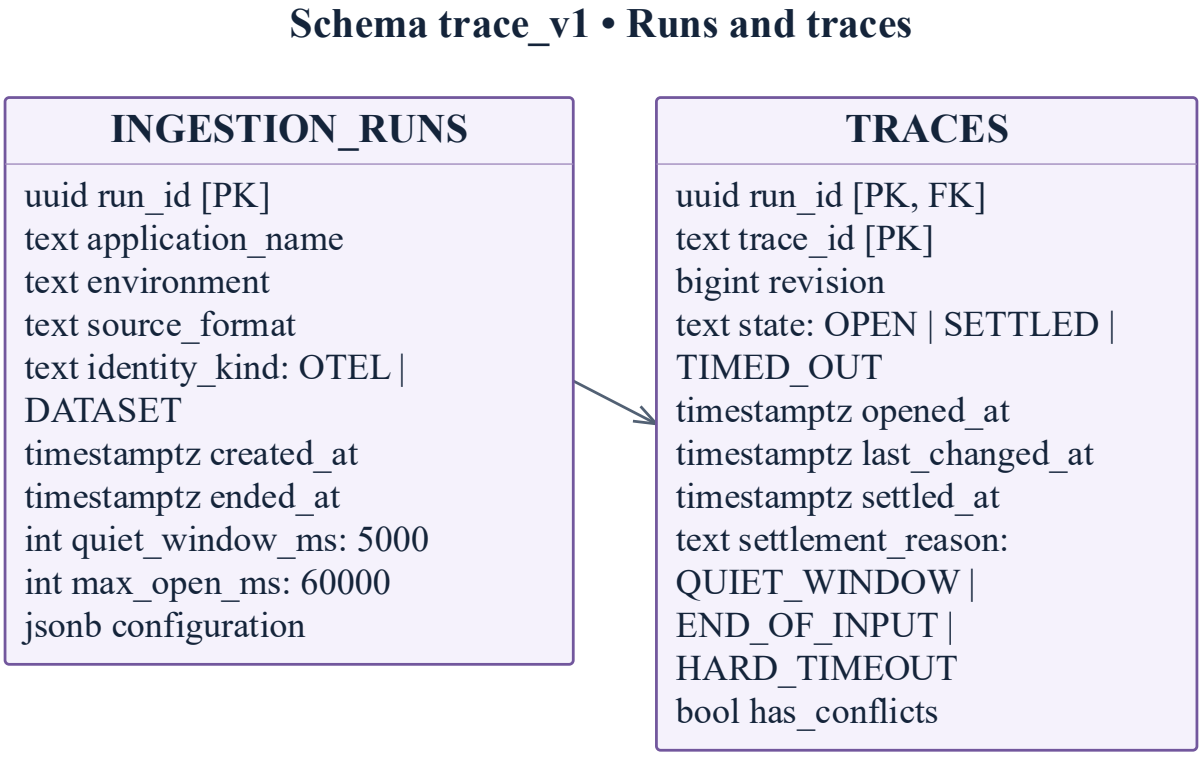


Fig. 7(a). Entity-relationship diagram of trace_v1: ingestion runs and their traces.

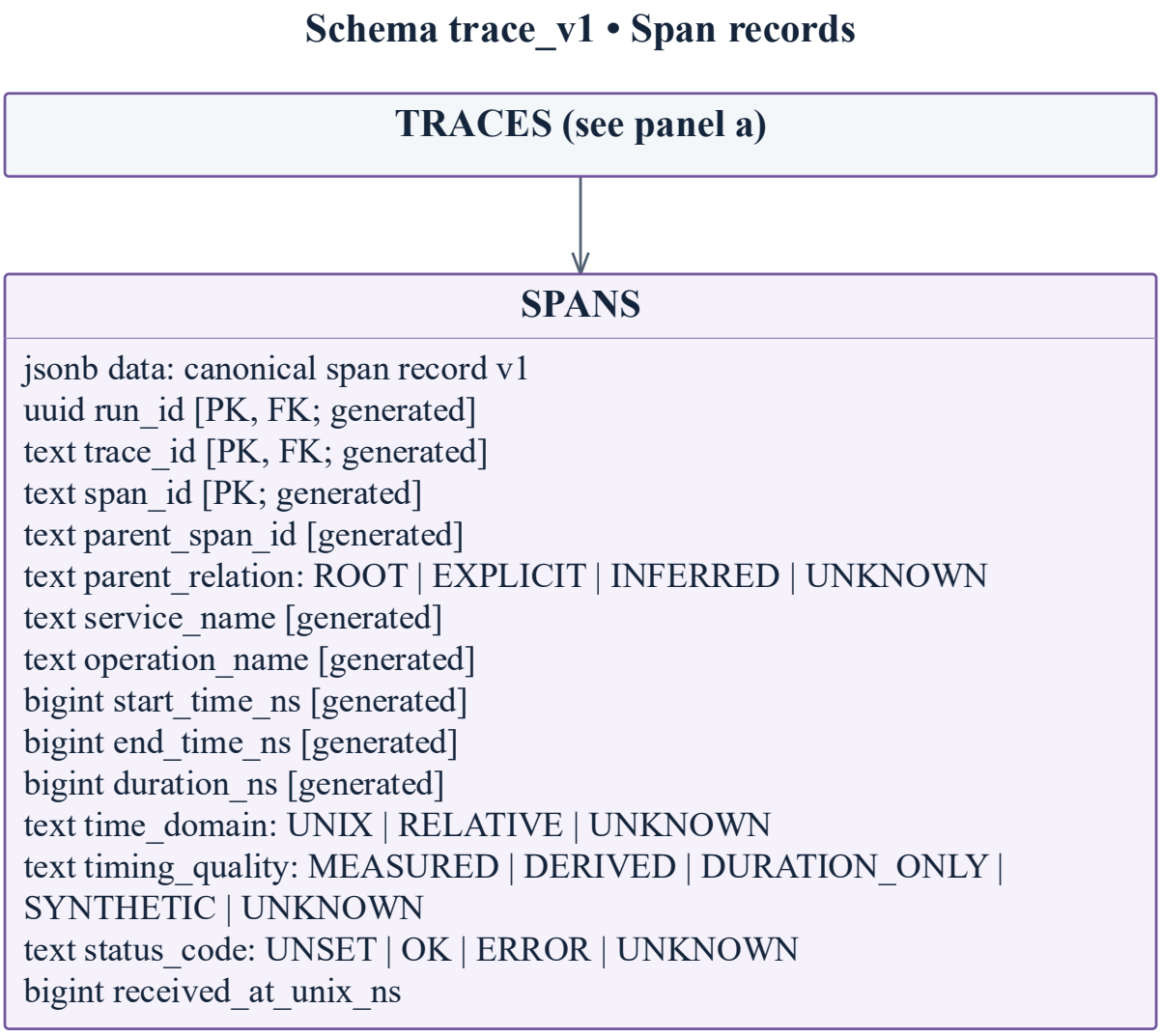


Fig. 7(b). Entity-relationship diagram of trace_v1: canonical span records belonging to each trace.

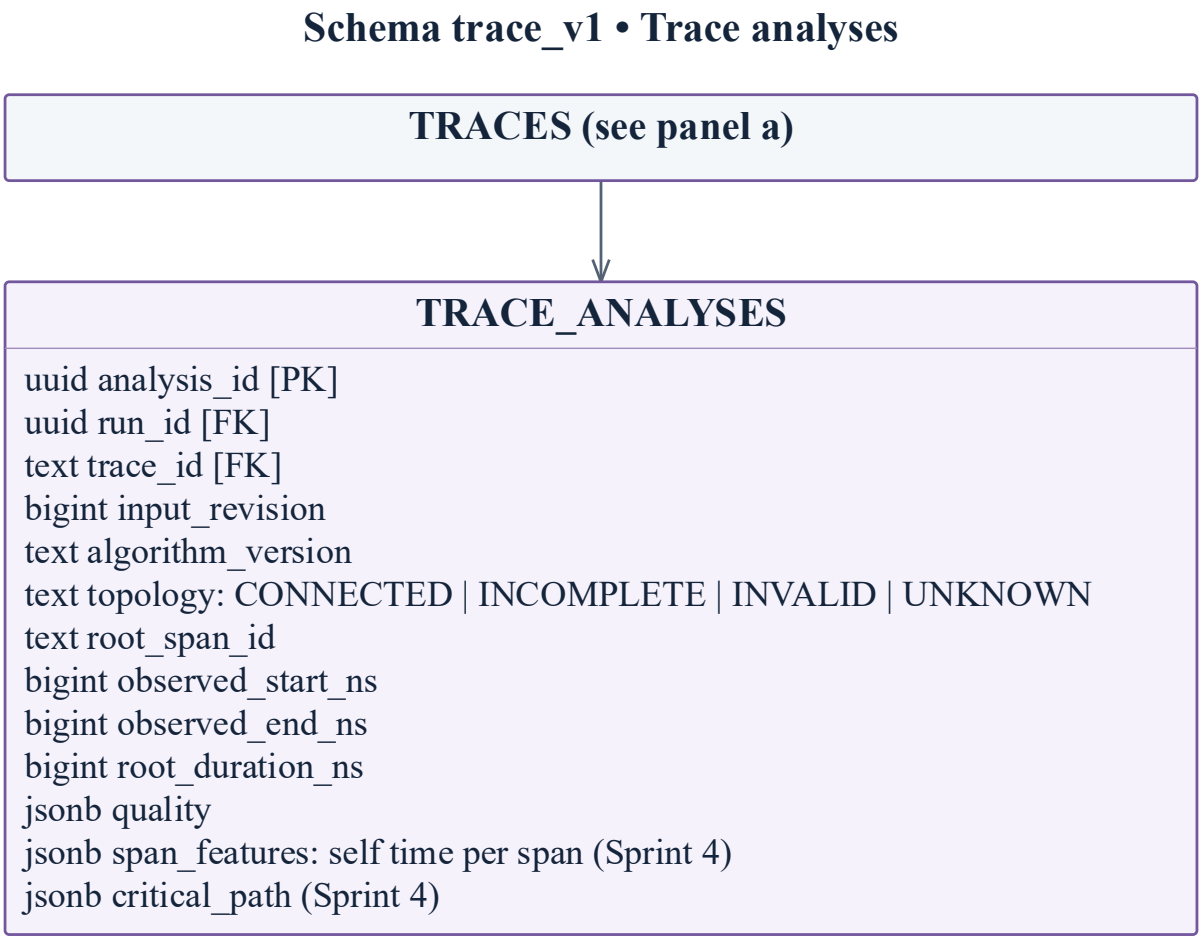


Fig. 7(c). Entity-relationship diagram of trace_v1: revision-specific trace analyses and diagnostic features.

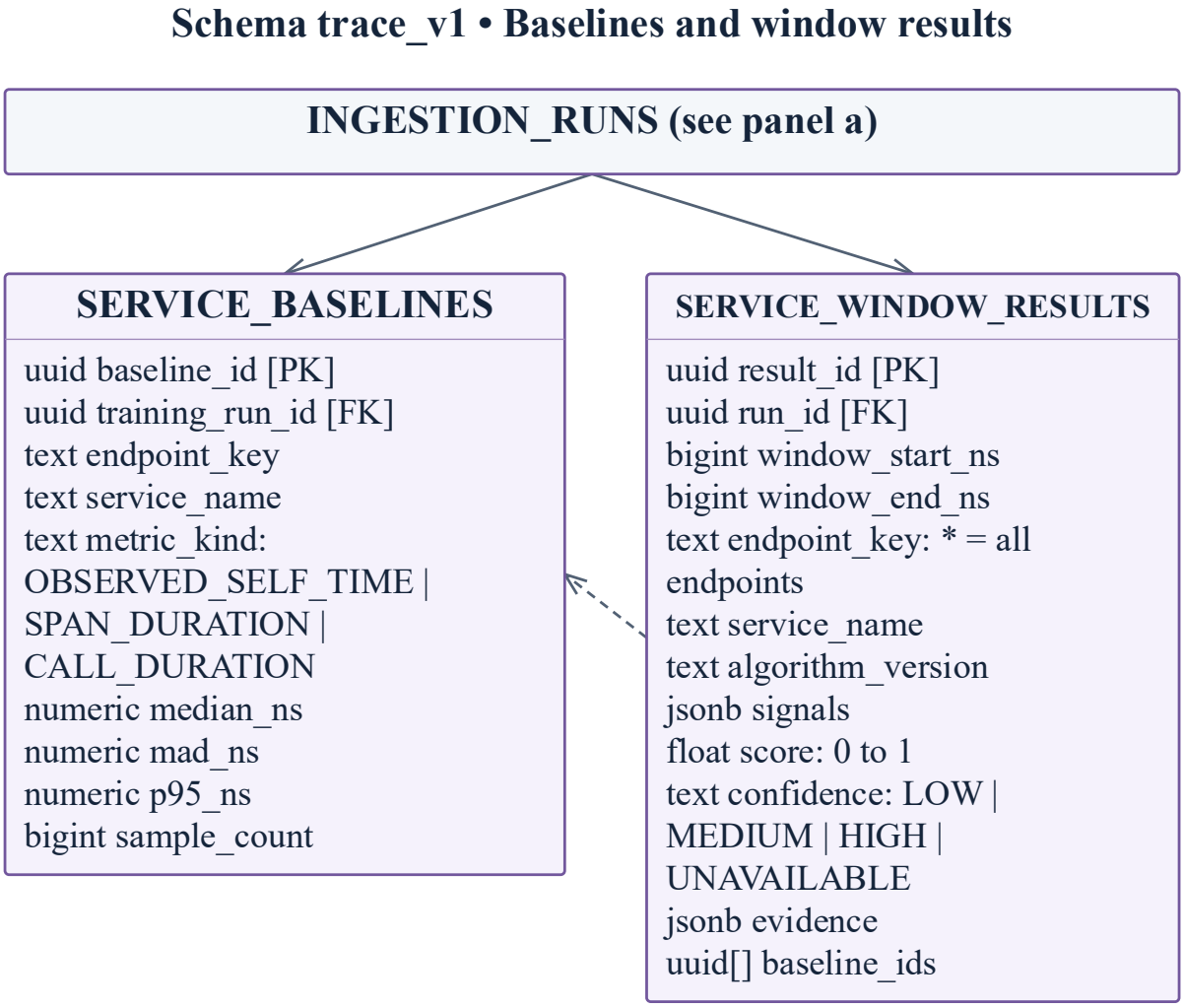
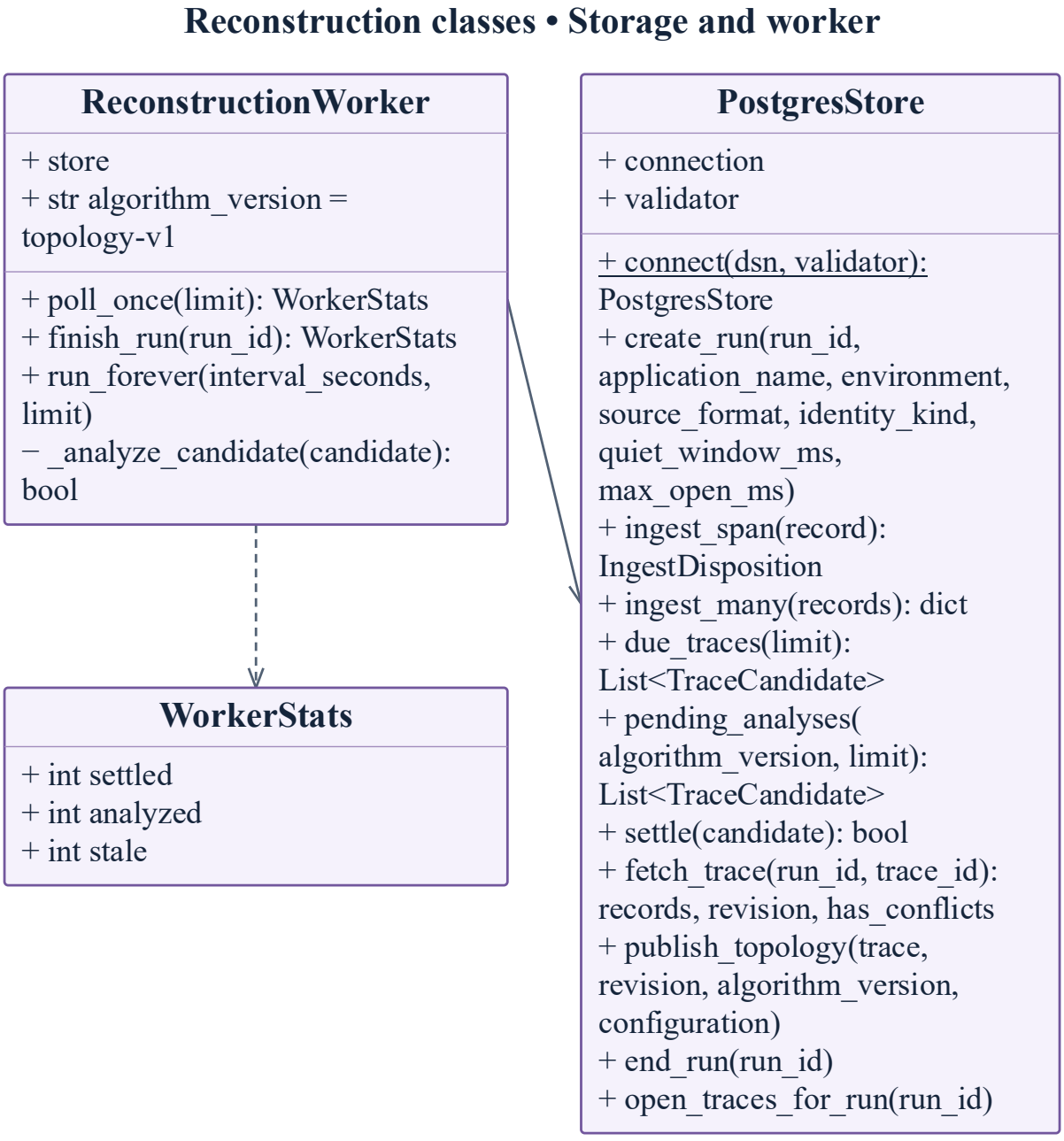


Fig. 7(d). Entity-relationship diagram of trace_v1: service baselines and scored analysis windows.

Fig. 8 shows the classes. PostgresStore wraps the SQL functions (ingest_span, settle_trace) and the candidate queries; topology.reconstruct_trace links parents, detects cycles and labels the trace CONNECTED, INCOMPLETE or INVALID; ReconstructionWorker runs the two-phase loop (settle due traces, then analyze settled traces whose current revision has no analysis). The worker is a plain class with poll_once and finish_run so it can run as a Lambda on a schedule, as a local process (python -m trace_reconstruction poll), or inside a test with an in-memory store.



The worker uses topology in panel (b). PostgresStore depends on TraceCandidate, IngestDisposition and StaleTraceError, also shown in panel (b).

Fig. 8(a). Reconstruction package class diagram: PostgresStore, ReconstructionWorker and WorkerStats.

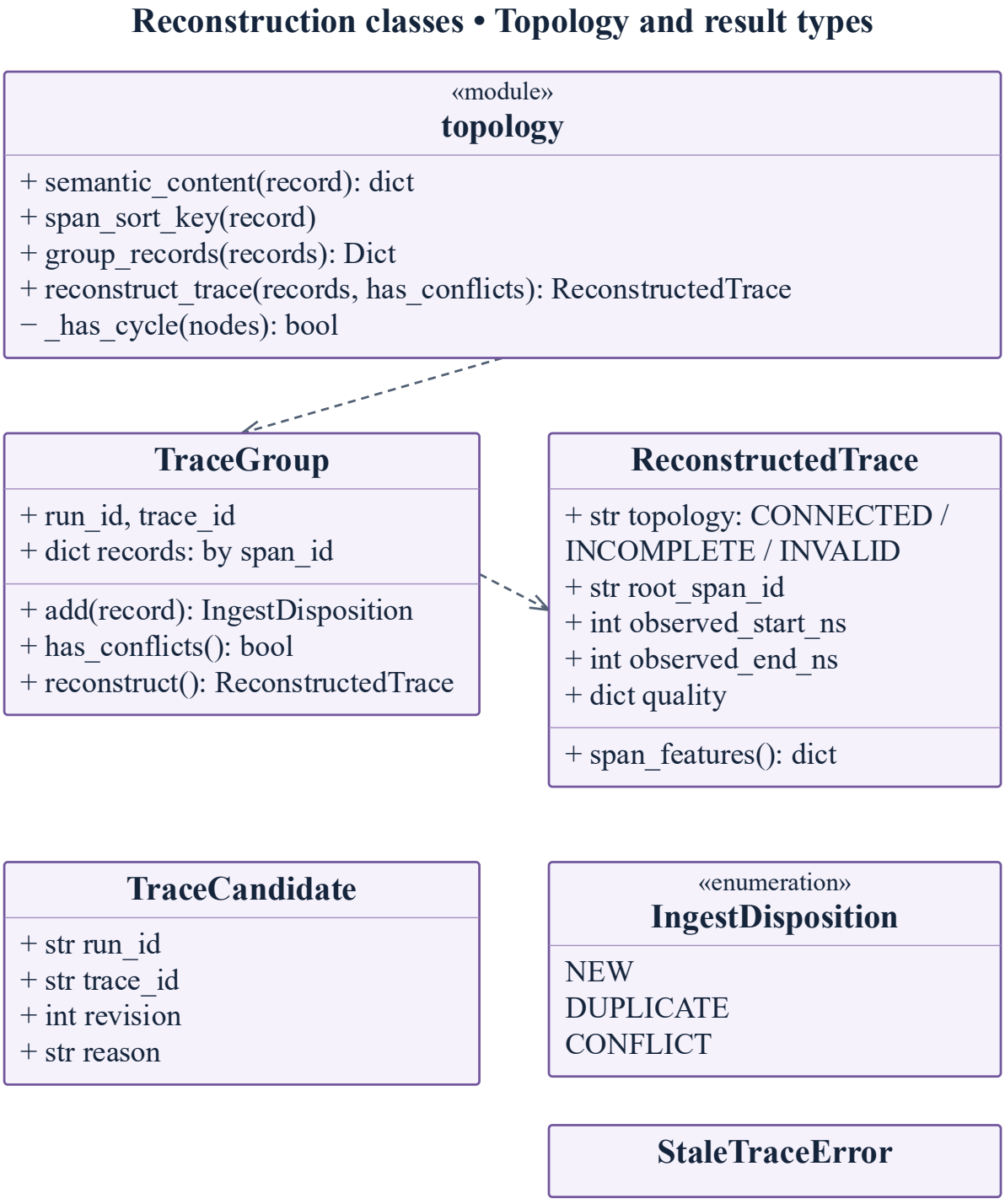


Fig. 8(b). Reconstruction package class diagram, continued: topology operations, reconstructed trace data and storage result types.

Analysis design: self time, critical path, slow-versus-normal, score
Three computations feed every diagnosis. They are implemented today in dashboard/app/analysis.py so the dashboard can show evidence from the spans directly; the diagnosis Lambda (Sprint 4–5) will compute the same values once per settled trace and store them in trace_analyses.span_features and critical_path, at which point the dashboard reads them instead of recomputing.

- Self time of a span = its duration minus the union of its children's intervals (overlapping children are merged first, so two parallel calls are not subtracted twice).
- Critical path = walk back from the root's end: the child that finished last is on the path, then from that child's start look for the child that finished before it, and so on (CRISP's rule). Sequential children all count; of overlapping children only the last one does. Each on-path span contributes its duration minus its on-path children, and the per-service shares add up exactly to the root duration (checked by a unit test).
- Slow-versus-normal split = traces of one endpoint over a threshold, or the slowest 10% when no threshold is given; with fewer than 10 traces there is no tail and everything is normal. For each service the compare view reports p50/p95 self time and critical-path share in both groups, the added milliseconds, and the ratio.

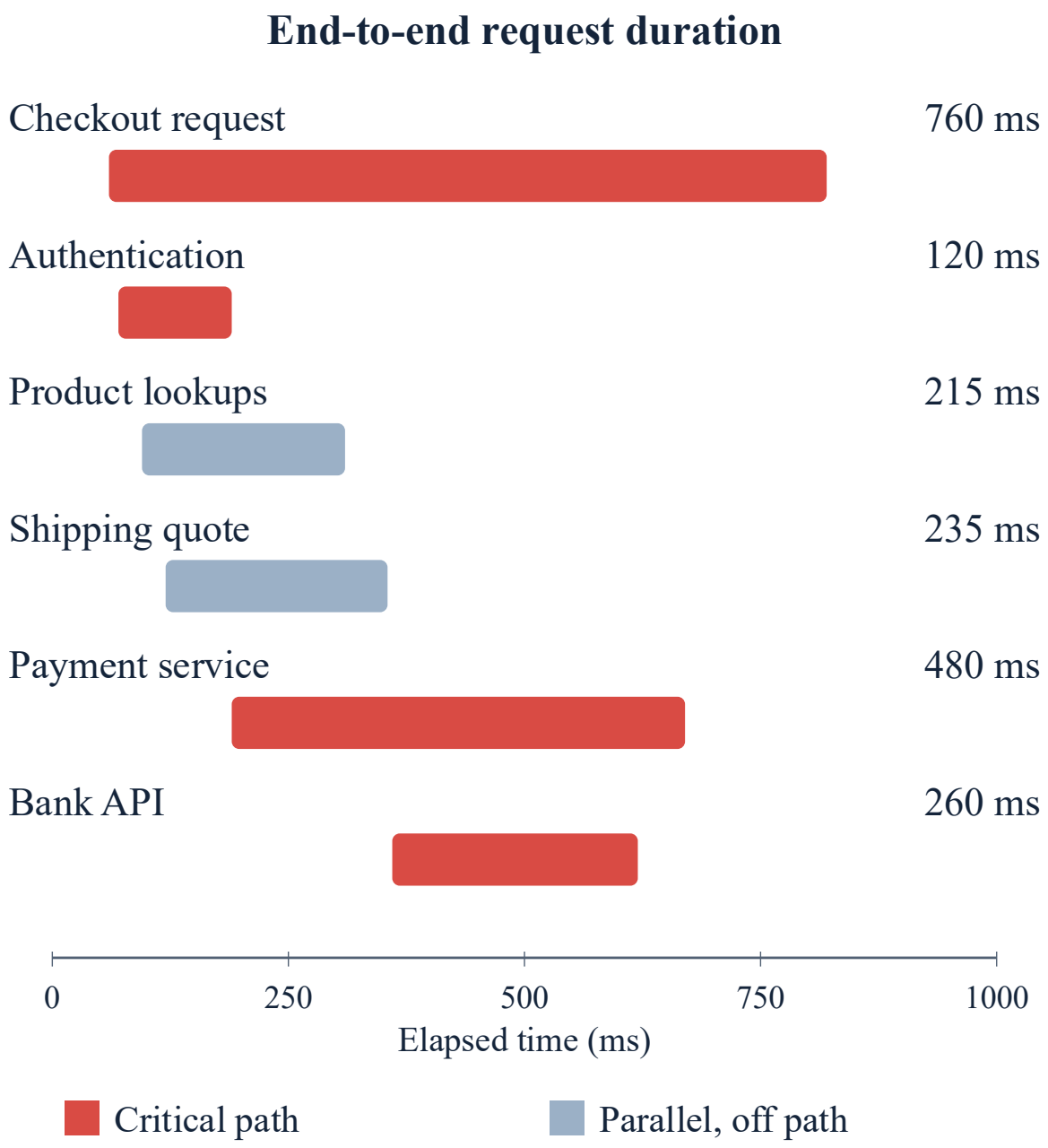


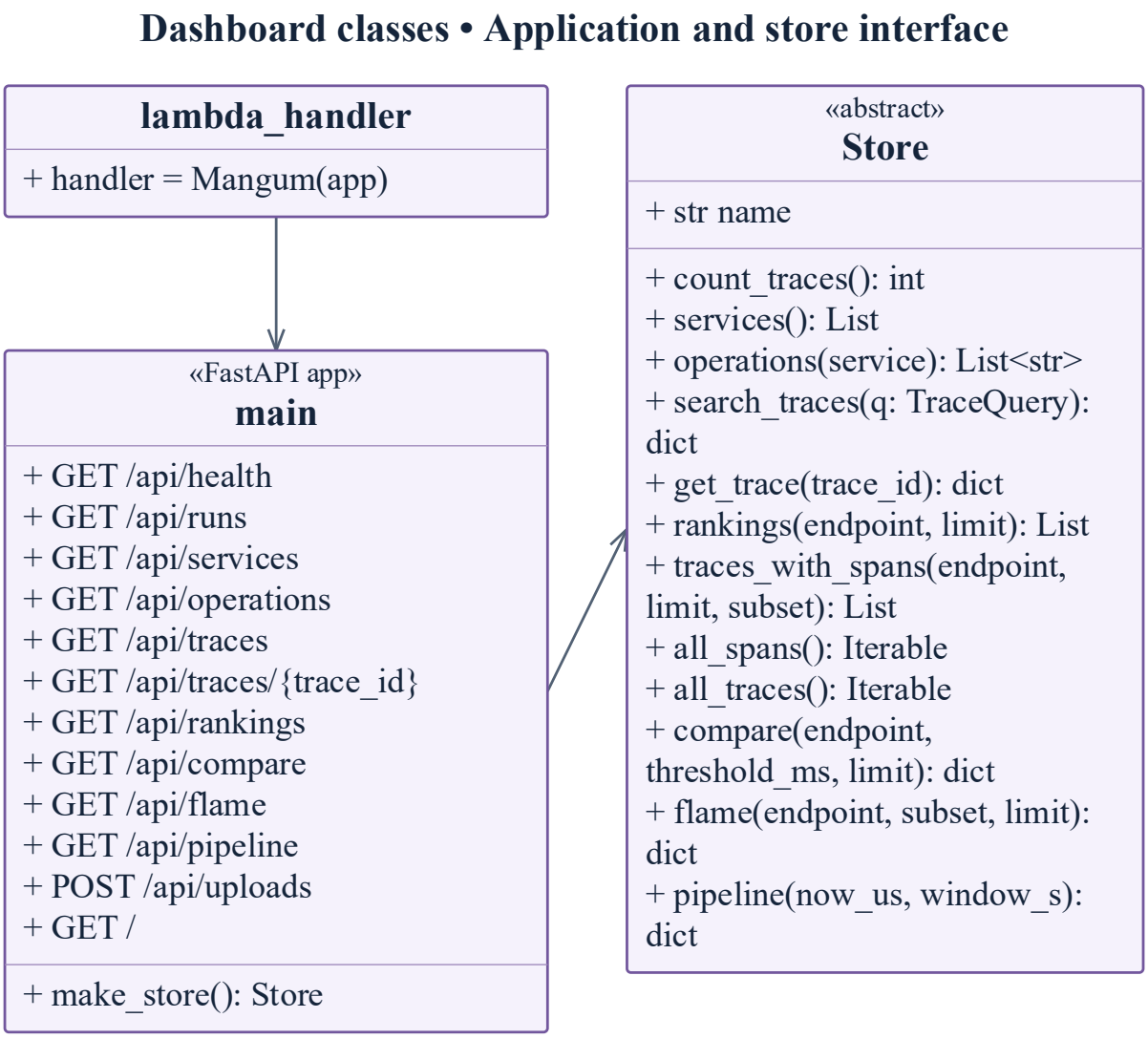
Fig. 9. Request-duration waterfall for one checkout trace. Red bars mark critical-path calls; blue bars show parallel product lookups and the shipping quote. Durations are in milliseconds.

Bottleneck score (Larry, Sprint 4–5). For each (endpoint, window) the diagnosis step computes per-service signals: excess critical-path milliseconds in slow traces versus the endpoint's reference window, deviation from the service's own baseline (median and MAD from service_baselines), slow-request frequency, error rate with sign discipline, and the two FIRM-

derived signals (relative importance and congestion intensity). Signals are rank-normalized, combined with weights that are tuned on the first two-thirds of the data by time and then frozen, and written to service_window_results with a score in [0, 1], a confidence (LOW/MEDIUM/HIGH, or UNAVAILABLE when the window has too few traces or the pipeline is lagging) and the evidence array the dashboard displays. The weights are a hypothesis: the ablation ladder in Chapter 7 decides whether anything beyond critical-path share helps.

Dashboard API and user interface design

The dashboard is a FastAPI service and one static HTML page (dashboard/static/index.html, plain JavaScript, no build step). Fig. 10 is the class diagram. The important design decision is the Store interface: the routes never touch SQL. JsonlStore reads normalized files from a folder (local work, demos, tests), SqlStore is a small SQLite layout for development, and TraceV1Store reads the reconstruction tables in PostgreSQL. All three return the same plain dictionaries, and the derived views (compare, flame, pipeline) are implemented once in the base class on top of traces_with_spans, so a new store only has to implement data access.
STORE=jsonl|sqlite|trace_v1 selects the store at start-up.



Store implementations appear in panel (b). main uses the response models in panel (c); Store uses TraceQuery and analysis from panel (c).

Fig. 10(a). Dashboard service class diagram: Lambda entry point, FastAPI application and abstract Store interface.

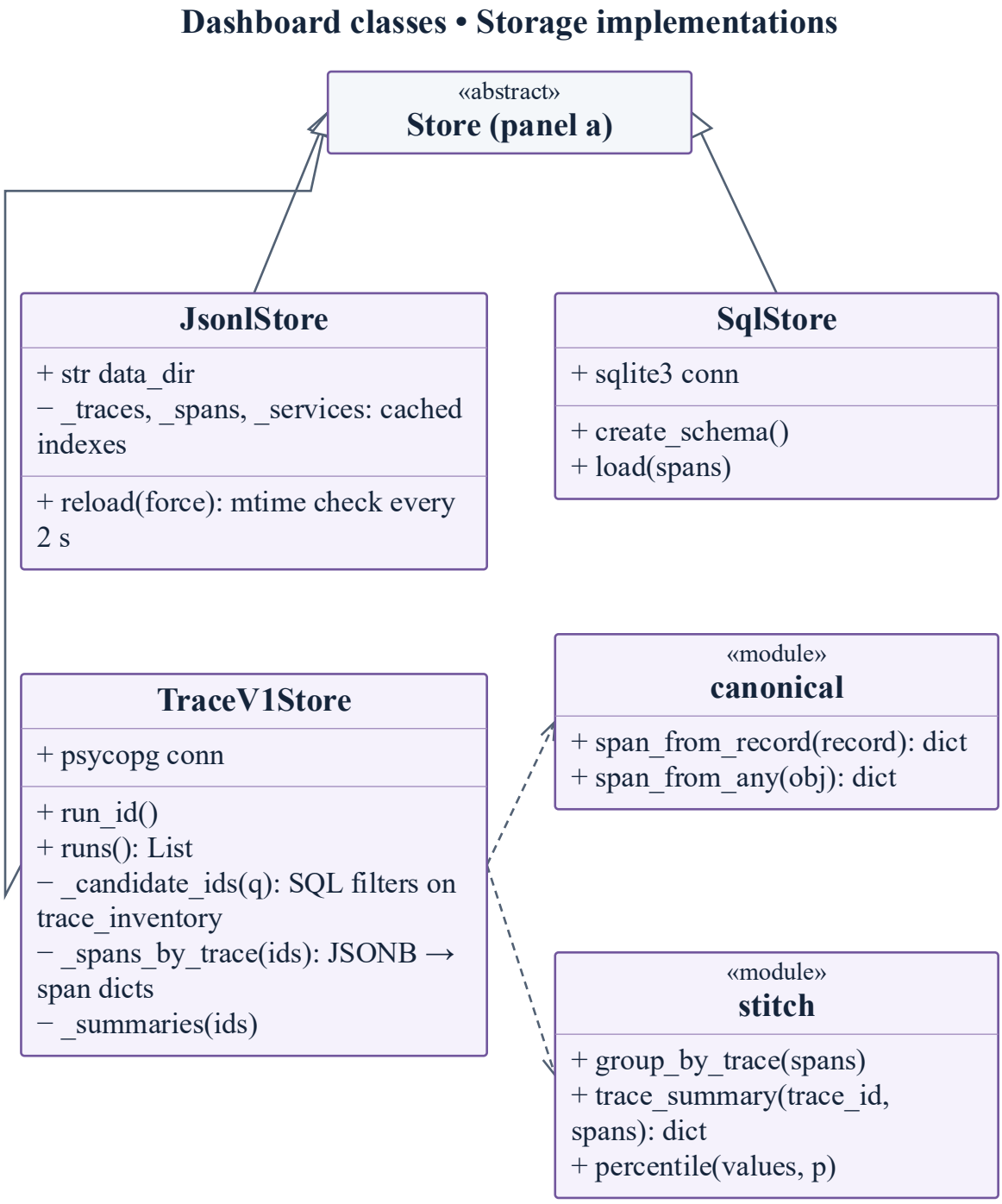
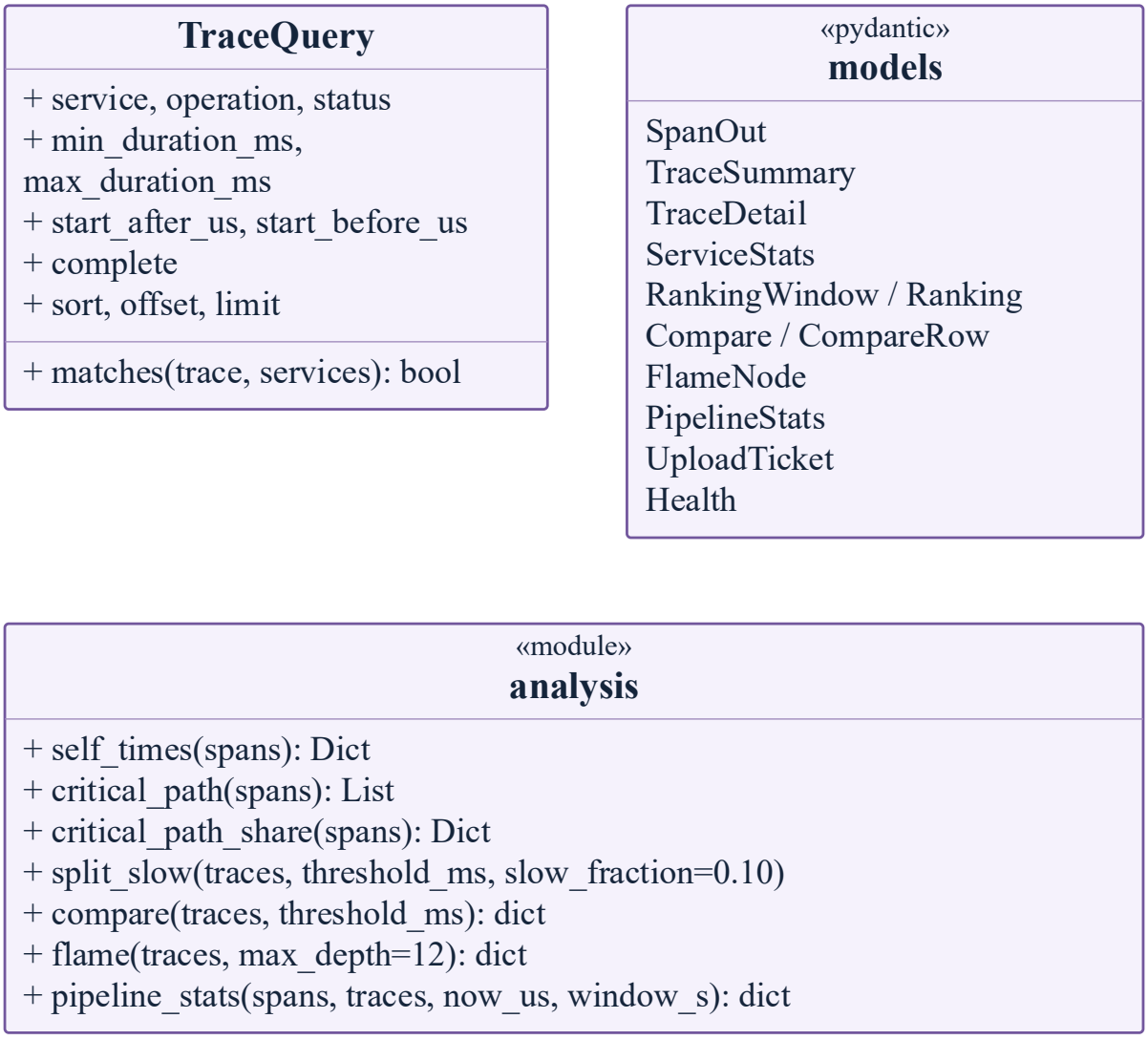


Fig. 10(b). Dashboard service class diagram, continued: Store implementations and their canonical and stitching helpers.

Dashboard types • Queries, analysis and responses



Store depends on TraceQuery and analysis. main depends on the response models. Those application classes are shown in panel (a).

Fig. 10(c). Dashboard service class diagram, continued: trace queries, analysis operations and response models.

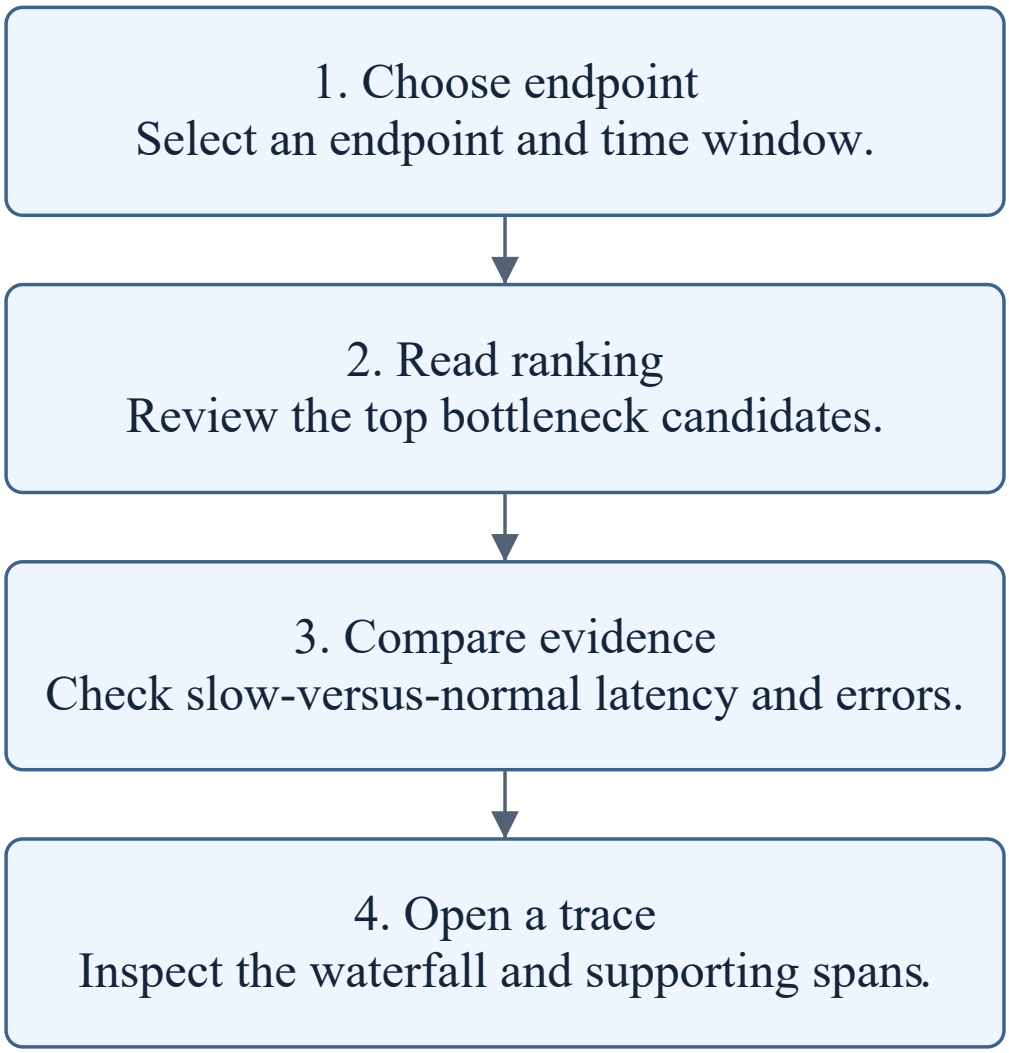
Route	Parameters	Returns	Backs
GET /api/health	—	store name, trace count	status pill
GET /api/runs	—	ingestion runs (trace_v1 only)	run selector in the header
GET /api/services	—	span/trace counts, error rate, p50/p99 per service	Overview table and bars
GET /api/operations	service	operation names	filter dropdown

Route	Parameters	Returns	Backs
GET /api/traces	service, operation, status, min/max_duration_ms, start_after/before_us, complete, sort, offset, limit ≤ 500	paged TraceSummary list + total	Traces view, search and filters (E8)
GET /api/traces/{id}	—	summary + spans with self time	waterfall, deep link #trace=…
GET /api/rankings	endpoint, limit	windows with ranked services, score, confidence, evidence, note	Bottlenecks view (E6, D3)
GET /api/compare	endpoint, threshold_ms, limit	per-service slow vs normal p50/p95, added_ms, ratio	Slow vs normal table
GET /api/flame	endpoint, subset=all|slow|normal, limit	merged tree, value = self time	Flame graph view (D1)
GET /api/pipeline	window_s	staleness p50/p95/p99, lag, ingest rate, incomplete rate, series	Pipeline health view (D4, N1)
POST /api/uploads	filename, content_type	presigned PUT URL, key, expiry	Upload view (E3)

TABLE III

DASHBOARD API. ALL RESPONSES ARE PYDANTIC MODELS (DASHBOARD/APP/MODELS.PY), SO THE OPENAPI SCHEMA AT /DOCS IS GENERATED FROM THE CODE

Fig. 11 is the sequence of one investigation: the engineer picks an endpoint, reads the ranking, checks the slow-versus-normal numbers, and opens a slow trace. Requirement E8 asks for slow episode to ranked list with supporting traces in under five clicks; this flow is three.



The dashboard investigation flow uses four steps.

Fig. 11. Dashboard investigation workflow: select an endpoint and time window, review the bottleneck ranking, compare latency and error evidence, and inspect a trace.

UI screenshots. The following figures show selected details from the running dashboard on demo data (tools/make_demo_data.py). Fig. 12 shows service p50/p99 latencies, slow traces, and the latest bottleneck ranking. Fig. 13 shows a selected checkout trace, its waterfall divided into three enlarged time segments, and selected span-table columns. Service labels repeat beside each segment; self time is marked inside the bars. Fig. 14 shows bottleneck evidence and the slow-versus-normal comparison. Fig. 15 shows the merged flame-graph controls and enlarged left and right segments. Fig. 16 shows freshness percentiles, pipeline health, and normalization throughput. The crops omit navigation, unused space, and selected columns to improve readability.

Service	p50	p99
cartservice	4.9 ms	6.5 ms
checkoutservice	52.9 ms	312.1 ms
emailservice	4.0 ms	6.2 ms
frontend	75.9 ms	334.4 ms
paymentservice	20.5 ms	280.4 ms
productcatalogservice	4.0 ms	6.7 ms
quoteservice	7.2 ms	10.1 ms
redis	1.0 ms	2.1 ms
shippingservice	13.2 ms	17.1 ms

Fig. 12(a). Dashboard overview: service names and p50/p99 latency columns, cropped and placed side by side.

Slowest traces

Trace	Endpoint	Duration	Status
d3c6e9c0ea6ef9…	HTTP POST /cart/checkout	361.4 ms	ok
88f59127eb86d8…	HTTP POST /cart/checkout	340.2 ms	ok
7fac6531905011…	HTTP POST /cart/checkout	334.4 ms	ok
d50049b7c8b7ae…	HTTP POST /cart/checkout	327.7 ms	ok
9e510261d41f59…	HTTP POST /cart/checkout	325.8 ms	ok
1c97f2b04f4224…	HTTP POST /cart/checkout	314.3 ms	ok
81eb5624fdce75…	HTTP POST /cart/checkout	312.8 ms	ok
be44f4901d6337…	HTTP POST /cart/checkout	310.7 ms	error

Fig. 12(b). Dashboard overview: slowest traces, endpoints, durations, and status.

Latest bottleneck ranking

10/22/2026, 9:05:22 PM → 10/22/2026, 9:10:22 PM · HTTP POST /cart/checkout · bottleneck_score · score gap 76% (no confidence reported)

#1	paymentservice	0.91
#2	checkoutservice	0.22
#3	shippingservice	0.08

Fig. 12(c). Dashboard overview: the latest bottleneck ranking and service scores.

Trace `d3c6e9c0ea6ef93387ae9ad58c7fd3a9`

Endpoint	frontend · HTTP POST /cart/checkout
Start	10/22/2026, 9:07:21 PM
Duration	361.4 ms
Spans	11 across 9 services
Status	ok
Source	otlp

Fig. 13(a). Trace detail: selected checkout request, duration, span count, and source.

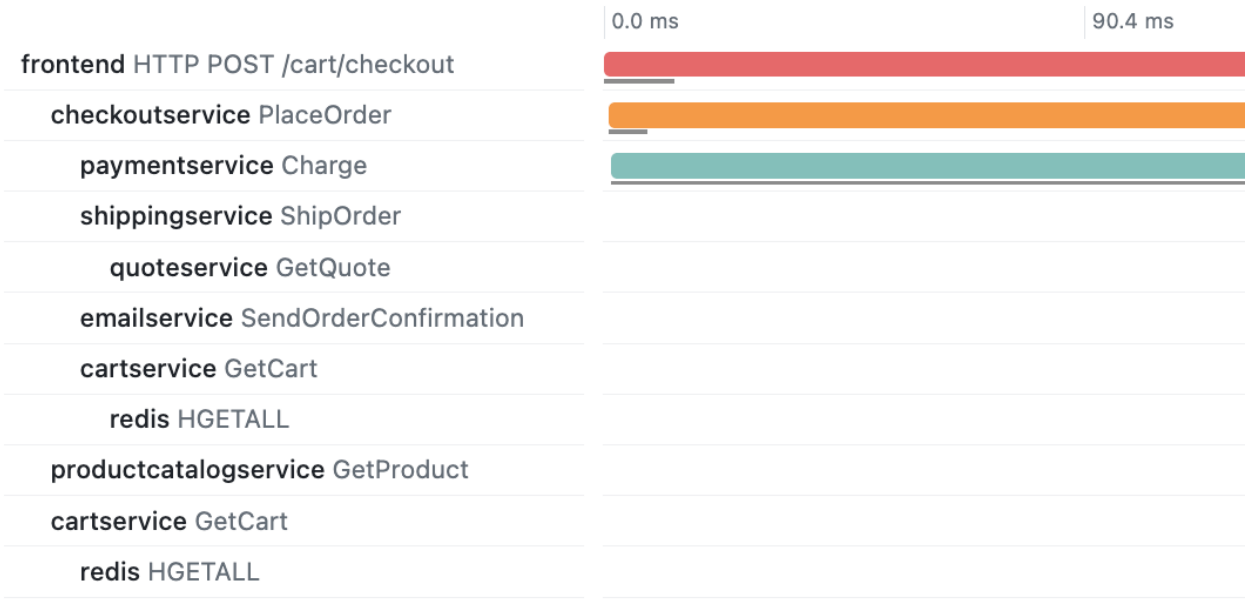


Fig. 13(b). Trace waterfall: early portion of the time axis, with service labels at left. The timeline continues in panels (c) and (d).

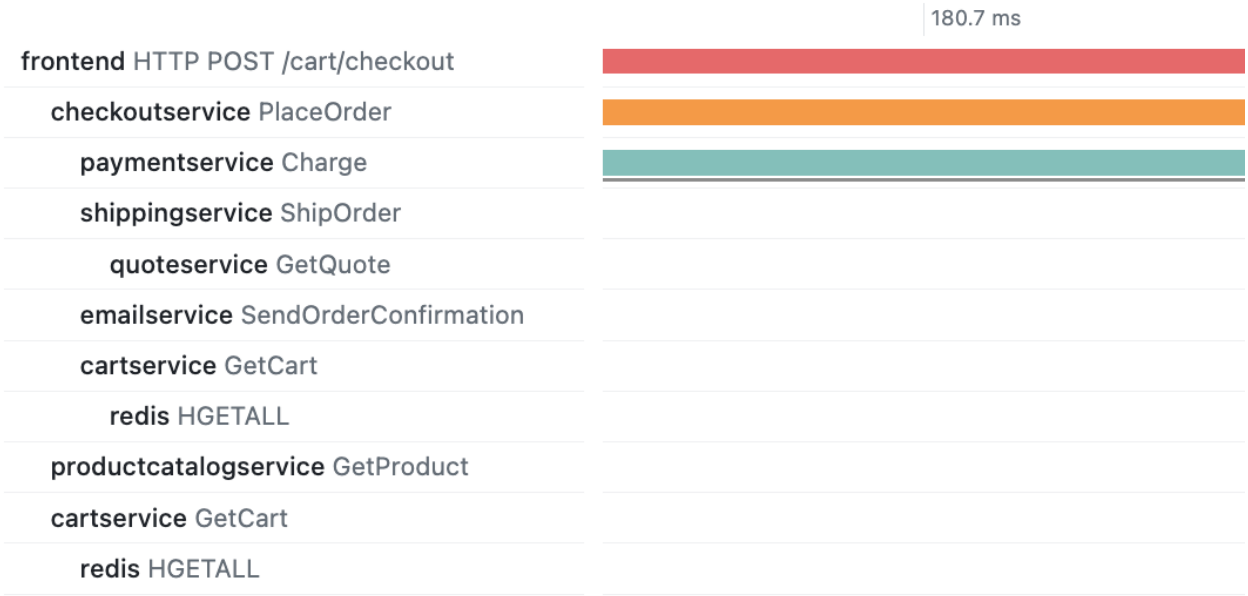


Fig. 13(c). Trace waterfall: middle portion of the time axis, with service labels repeated at left.

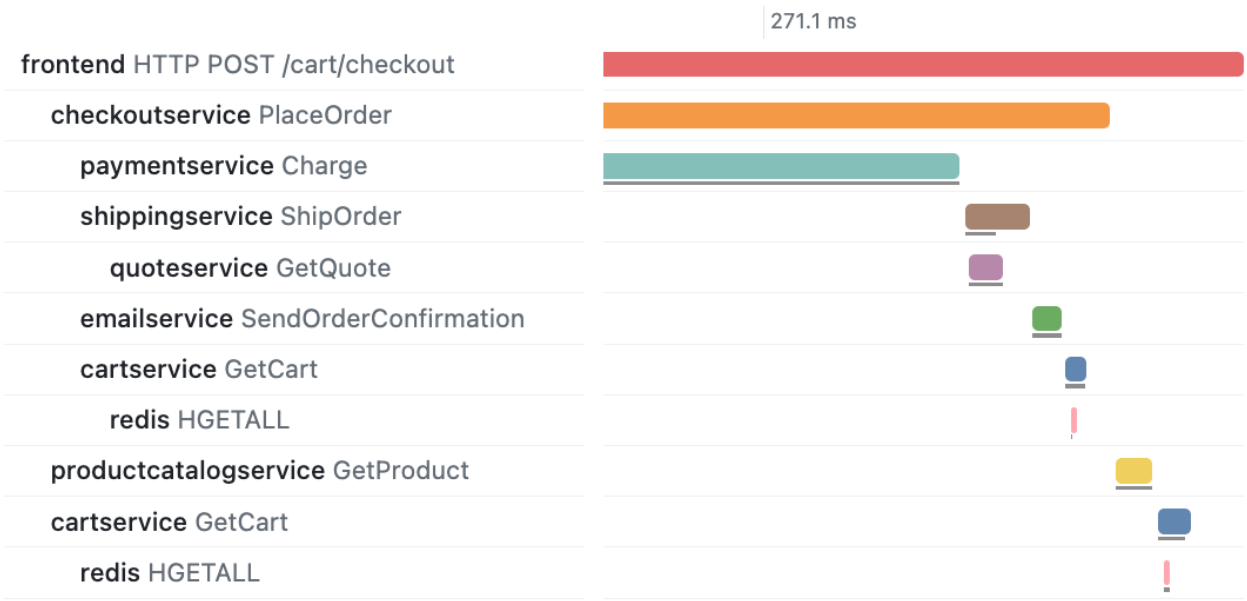


Fig. 13(d). Trace waterfall: late portion of the time axis, with service labels repeated at left.

Service	Operation	Duration	Self
frontend	HTTP POST /cart/checkout	361.4 ms	13.2 ms
checkoutservice	PlaceOrder	335.2 ms	7.28 ms
paymentservice	Charge	306.5 ms	306.5 ms
shippingservice	ShipOrder	11.9 ms	5.63 ms
quoteservice	GetQuote	6.31 ms	6.31 ms
emailservice	SendOrderConfirmation	5.53 ms	5.53 ms
cartservice	GetCart	3.93 ms	3.63 ms
redis	HGETALL	0.30 ms	0.30 ms
productcatalogservice	GetProduct	6.74 ms	6.74 ms
cartservice	GetCart	6.23 ms	5.16 ms
redis	HGETALL	1.07 ms	1.07 ms

Fig. 13(e). Span table: service and operation columns alongside duration and self time; other columns omitted.



Fig. 14(a). Bottleneck evidence for the leading services in the latest analysis window. Horizontal bars are cropped.

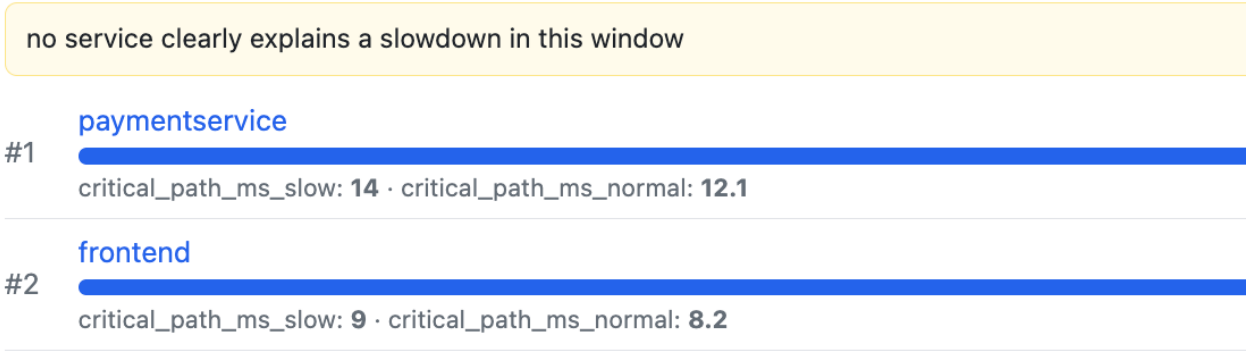


Fig. 14(b). Earlier analysis window: no service clearly explains the slowdown; cropped service evidence appears below.

Slow versus normal

Endpoint: all endpoints | Slow means longer than (ms): slowest 10% | Compare

20 slow traces (≥ 280.1 ms) versus 180 normal ones

Service	Self time, slow (p50)	Self time, normal (p50)	Added	Ratio
paymentservice	243.9 ms	15.4 ms	+228.5 ms	15.8×
productcatalogservice	4.6 ms	4.0 ms	+0.6 ms	1.1×
redis	2.1 ms	2.0 ms	+0.1 ms	1.1×
cartservice	7.9 ms	7.8 ms	+0.1 ms	1.0×
frontend	11.3 ms	11.3 ms	+0.1 ms	1.0×
shippingservice	6.0 ms	6.1 ms	-0.1 ms	1.0×
checkoutservice	9.6 ms	10.0 ms	-0.3 ms	1.0×
quoteservice	6.9 ms	7.2 ms	-0.3 ms	1.0×
emailservice	3.6 ms	4.0 ms	-0.4 ms	0.9×

Fig. 14(c). Slow-versus-normal comparison: service self time, added latency, and ratio. Right-hand columns omitted.

Flame graph

Endpoint: all endpoints | Traces: all | Most recent: 300 | Draw

Fig. 15(a). Flame-graph controls for endpoint, trace selection, and recent-trace limit.

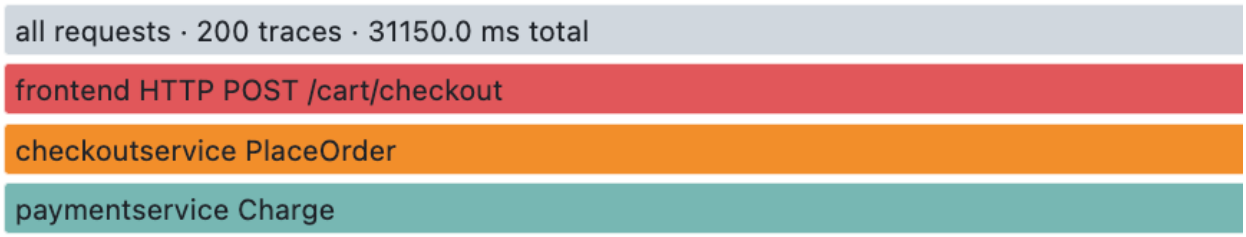


Fig. 15(b). Flame graph, left segment: merged request total and the frontend, checkout, and payment path.

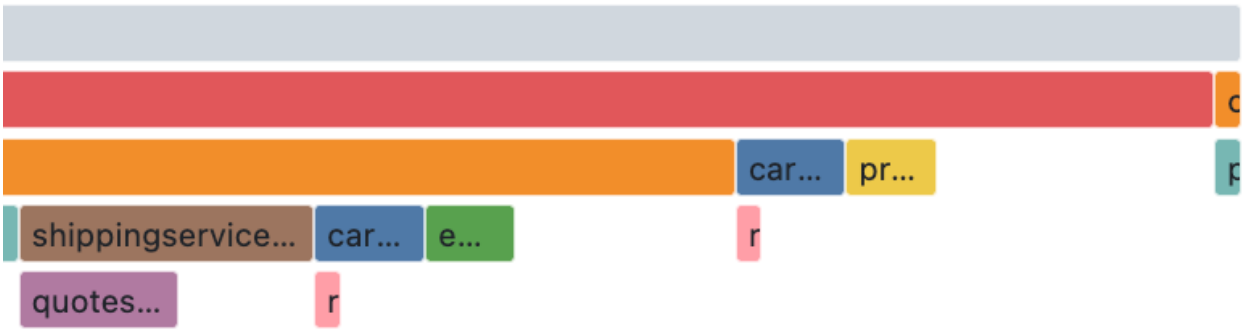


Fig. 15(c). Flame graph, right segment: smaller downstream branches. Truncated labels are retained from the app.

Freshness

Staleness p50	3.8 s
Staleness p95	5.8 s ok
Staleness p99	6.1 s
Adapter lag p50 / p95	1.4 s / 2.3 s
Last span normalized	10/22/2026, 9:10:26 PM (10 min ago)

Staleness is how old a span is when it becomes visible here (normalized time minus span end). Target: p95 under 60 s at 1x replay. Lag is the time between the raw file landing in S3 and the adapter writing it out.

Fig. 16(a). Pipeline freshness: staleness percentiles, adapter lag, and normalization time.

Health

Traces	200
Spans	2200
Spans with ingest stamps	2200 (100%)
Incomplete traces	1 (0.5%) ok
Ingest rate (last 15 min)	74.8 spans/min

Fig. 16(b). Pipeline health: trace and span counts, ingest-stamp coverage, incomplete traces, and ingest rate.



Fig. 16(c). Pipeline throughput: spans normalized per minute during the last 15 minutes.

Navigation is hash-based (#trace=…, #view=bottlenecks&endpoint=…) so a diagnosis can be pasted into a chat message and opens on the same screen. Tables are capped at 600 rows and the

API at 500 per page; the store caches its indexes and re-reads files only when their modification time changes, which took /api/services from 660 ms to about 4 ms on the demo set.

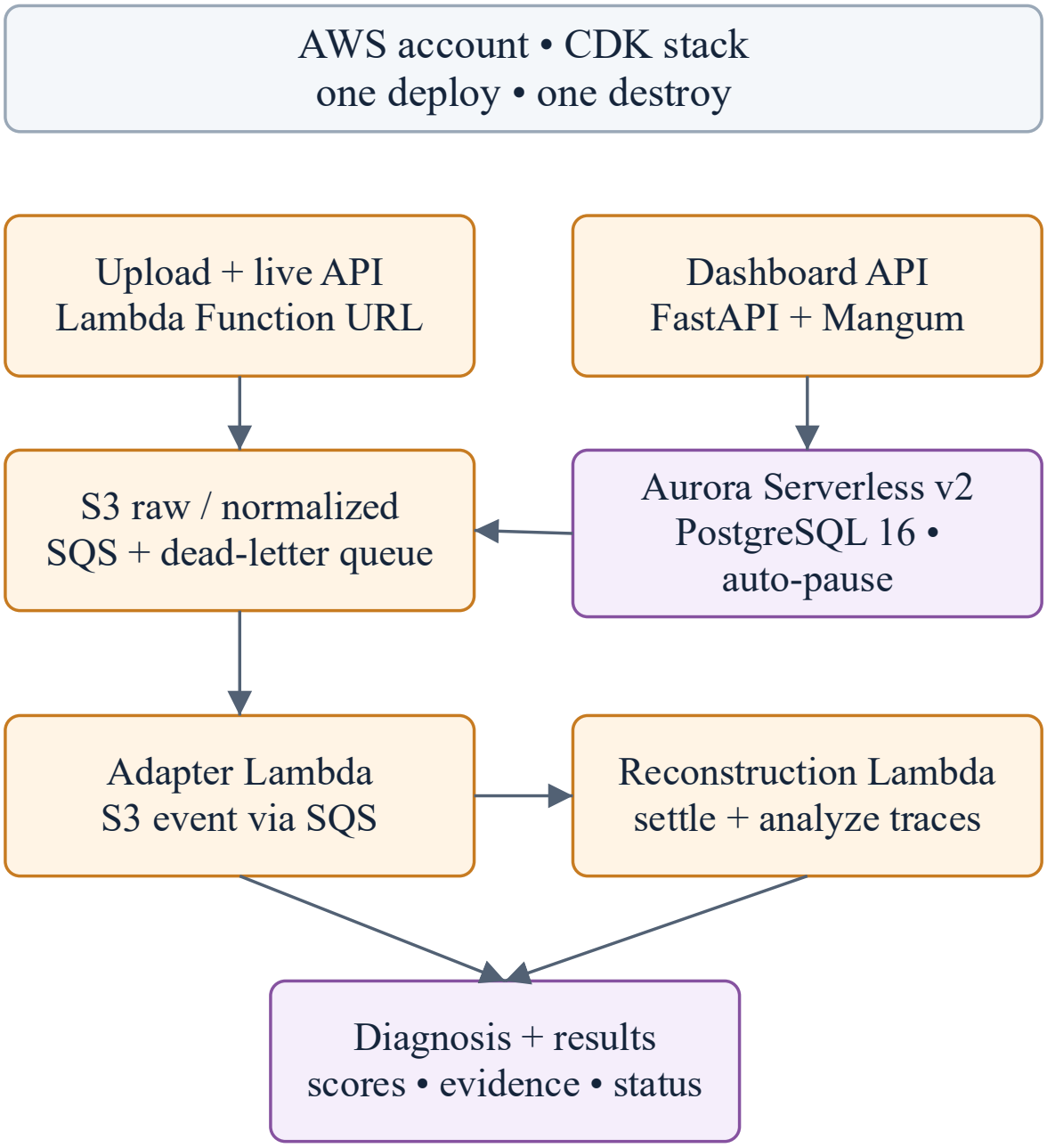
Evaluation harness design

The experiments package exists so every number in the report is produced by one command (requirement E10): python -m experiments.run experiments/config.json. Each step is skipped when its input is missing, so the same runner works from day one and grows as data arrives. The baselines module defines the ranking interface as a plain function, rank(normal_traces, anomalous_traces) → [service, …] best first, and ships the three naive heuristics from E9 (slowest average service, longest span in slow traces, critical-path-only). The proposed score and MicroRCA are added through extra_methods as module:function strings, so comparing a new method never requires touching the runner. The RCAEval evaluator reads each case folder with the ingest reader, splits traces at inject_time, asks every method for a ranking and records where the labelled service landed; metrics.py turns that into accuracy@1, accuracy@3 and MRR, overall and per fault type. figures.py draws the report charts, and the usability folder holds the protocol and the timing log for the Jaeger-versus-dashboard study.

Cloud deployment design

Contributed by: Matthew Tang (drafted from the architecture and the cost plan; to be reviewed by Matthew)

Fig. 17 is the deployment diagram. Everything lives in one AWS account and one CDK stack with one deploy and one destroy command (requirement N5). Lambdas run on arm64 with 256–512 MB; the adapter and reconstruction functions are triggered by S3 events through SQS so that bursts queue instead of failing; a dead-letter queue catches poison objects; the dashboard API is a Function URL in front of FastAPI through Mangum; Aurora Serverless v2 runs PostgreSQL 16 at 0.5–2 ACU with auto-pause. Configuration is environment variables only (STORE, DATABASE_URL, RUN_ID, RAW_BUCKET, RAW_PREFIX, UPLOAD_EXPIRES, INGEST_TOKEN), and secrets come from Secrets Manager rather than the repository (requirement N6).



Configuration is environment-based; secrets are held in Secrets Manager.

Fig. 17. Deployment diagram for the AWS environment.

Design trade-offs

Decision	Alternatives considered	Why we chose it	Cost of the choice
SQS between S3 and the adapter	Kinesis Data Streams (Workbook 1 plan); direct S3 → Lambda	No shard-hour cost when idle; retries and dead-letter queue are built in; a file is the natural unit of work for uploads	No ordering guarantee and no replay of the stream; ordering is handled by the reconstruction revisions instead
Lambda Function URLs	API Gateway HTTP API	No per-request charge, 6 MB payload is enough for OTLP batches, one less resource	No built-in auth or throttling; the collector checks the token itself and the dashboard URL is the only public one
JSONB record + generated columns	Fully normalized span columns	The record can evolve without migrations; the fields queries need are still indexed	~2x storage per span; acceptable with tail sampling and 30-day S3 lifecycle
Append-only result tables with revisions	UPDATE in place	Late spans and re-analysis never corrupt earlier results; every result is reproducible from its input_revision	More rows; views pick the current one
Analysis computed in the dashboard for now, precomputed later	Wait for the diagnosis step	The dashboard could show evidence and be tested weeks earlier; the same functions move to the diagnosis step	Two implementations until the switch; tests pin the results so they must agree
Plain HTML + JavaScript page	React/Vite single-page app	No build step, one file served by the API Lambda, easy for everyone to edit	Hand-written DOM code; fine at this page size
Microsecond integers inside the dashboard, nanosecond strings in the schema	One unit everywhere	JavaScript cannot hold ns safely; the schema must not lose precision	One conversion in canonical.py, covered by tests
Presigned uploads	Upload through the API	Large files, no Lambda payload limit, no cost for bytes passing through compute	Browser must do a second request; CORS rule on the bucket

TABLE IV

MAIN DESIGN TRADE-OFFS

Chapter 7. QA, Performance, Deployment Plan

Contributed by: Anukrithi Myadala (test strategy, test definitions, evaluation methodology, performance plan, reproduction steps), Larry Nguyen (reconstruction tests), Matthew Tang (deployment plan, bill of materials)

Test strategy

We test at five levels. Unit tests cover each package in isolation with fixtures checked into the repository; contract tests validate every record we write against the JSON Schema and the database constraints; integration tests run the dashboard store against a real PostgreSQL with the team schema; an end-to-end check runs a converted dataset through the adapter, the reconstruction worker, PostgreSQL and the dashboard API on one machine (compose.yaml); and browser smoke tests drive the page with Playwright. Table V lists what exists today and what each suite established. All suites run with the standard library test runner (python -m unittest) so nobody needs extra tooling, and the database and browser suites skip themselves when their dependency is absent rather than failing.

Suite (location)	Tests	What it checks	Status, Oct 22
ingest/tests	52	Alibaba dedupe and parent derivation; RCAEval CSV/Parquet reading and case labels; OTLP in and out; format detection; replayer ordering, speed and rebase (FakeClock); sinks and retries; upload ticket; adapter Lambda end-to-end on an S3 fixture, idempotent on re-run	all pass
trace_schema (validate.py, check_schema.py, check_ingestion.py)	contract	every example and every adapter output validates; schema.sql installs cleanly; ingest_span returns NEW / DUPLICATE / CONFLICT as specified	pass
reconstruction/tests	24	grouping and first-observation-wins dedupe; explicit and inferred parents; cycle detection → INVALID; quiet-window, end-of-input and hard-timeout settlement; revision bump on late span; worker poll loop; Astronomy Shop smoke fixture	all pass
dashboard/tests	76 (6 need PostgreSQL, 3 need a browser)	self time with overlapping children; critical path and that shares sum to root duration; slow/normal split rules; compare and flame outputs; pipeline stats from stamps; canonical record → span; all API routes and parameter validation (422 on bad input, 404 on unknown trace); request timing log and password-free start-up line; JSONL, SQLite and trace_v1 stores return identical answers on the same data; browser: page loads, search filters, waterfall renders	all pass (DB suite passes on PostgreSQL 16 with the team schema)
experiments/tests	11	metrics (accuracy@k, MRR) on hand-computed cases; synthetic RCAEval cases in the real layout (used only by the tests); baselines return every service exactly once; parallel workers give the same answer as one; runner writes results and summary	all pass

TABLE V

TEST SUITES IN THE REPOSITORY (163 AUTOMATED TESTS)

Test definitions

The tables below define the tests that matter most for the requirements in Chapter 3, with their expected outcome. Where the test already exists its name is given; planned tests are marked.

Req.	Test	Input	Expected outcome	Exists
E2	test_replay.test_emits_in_end_time_order_one_span_per_call	4 nested spans	spans leave in end-time order, never before their own end	yes
E2	test_replay.test_speedups	same spans at 1x, 2x, 10x	FakeClock sleep totals scale by 1/speed	yes
E2	test_alibaba.test_duplicate_keeps_caller_side_time	same (traceid, rpcid) twice	one span, larger (caller) rt and earliest start kept	yes
E3	test_adapter_lambda.test_idempotent	S3 fixture CSV, handler run twice	same canonical JSONL both times except the normalized stamp	yes
E3	test_upload.test_presigned_put / test_presign_endpoint	filename request to /api/uploads	sanitized key, PUT method, expiry = UPLOAD_EXPIRES	yes
E4	test_worker.test_stale_revision_is_not_published	settled trace + new span before analysis	revision bumped, stale analysis not published	yes
E4	test_topology.test_parent_cycle_is_invalid	A→B→A parents	topology INVALID, no critical path	yes
E4	test_analysis.test_share_per_service_adds_up_to_root	waterfall fixture	sum(shares) == root duration	yes
E6	test_tracev1_store.test_rankings_from_window_results	3 window rows incl. UNAVAILABLE	ranked by score, note set when nothing stands out	yes
E8	test_api.test_search_and_validation	service + min_duration + sort	only matching traces, total correct, limit ≤ 500 enforced	yes
E8	test_ui.test_trace_detail_and_deep_link	demo data, Playwright	waterfall shows N bars, deep link restores it	yes
E9/E12	test_experiments.test_summary	hand-computed ranks	accuracy@1, accuracy@3 and MRR match by hand	yes
N4	test_topology.test_exact_retry_ignores_arrival_metadata	same span delivered twice with different arrival metadata	one record, DUPLICATE disposition	yes
N7	test_analysis.test_stats	3 traces, 1 incomplete	incomplete_rate 0.33 and excluded from rankings	yes
E1	collector rejects bad token	POST without / with wrong token	401 / 403, nothing written to S3	planned (Matthew, Sprint 4)
E5	tail keep/drop keeps every slow trace	100 traces, 10 slow, 5% normal	all 10 slow kept, ~5 normal kept	planned (Larry, Sprint 5)
E11	Astronomy Shop fault flags	paymentServiceFailure on	payment ranked in top 3	fixture exists; full run planned (Sprint 5)

TABLE VI

REQUIREMENT-LEVEL TEST DEFINITIONS

Evaluation methodology

The evaluation follows the plan agreed with our advisor in September and recorded in Workbook 1 (RQ1–RQ4). This section gives the procedure, the baselines that are implemented, and what has been run so far.

Datasets. (1) Alibaba 2021 microservice call-graph traces: real production data, used for the streaming and explanation-validity claims; we use one 12-hour window, replayed through the pipeline. (2) RCAEval (RE2 online-boutique and train-ticket cases): labelled failure cases with the injected service and time, used for accuracy. (3) OpenTelemetry Astronomy Shop with its built-in failure flags, used as a smoke test where the root cause is known.

Baselines. Three naive heuristics are implemented in experiments/baselines.py: slowest average service, longest span in slow traces, and critical-path-only ranking. Manual diagnosis in Jaeger is set up in baselines/jaeger (same OTLP stream, docker compose) with a written protocol for the timed comparison. MicroRCA (personalized PageRank on an anomaly-weighted service graph) is scheduled for Sprint 5 and plugs into the runner through the same interface; BARO is a second published candidate if time allows.

Metrics. Accuracy@1, accuracy@3 and MRR on RCAEval cases, overall and per fault type (delay, error, CPU). Explanation validity on Alibaba episodes: remove the flagged service's attributed critical-path time and measure how much of the slow-versus-normal gap disappears. Freshness: p50/p95/p99 of (dashboard-visible time − trace completion time), target p95 < 60 s at 1x. Throughput: spans/s at 1x, 2x, 10x with zero drops. Cost: measured dollars from the AWS bill per configuration, and cost per million spans.

Procedure and discipline. Signals and weights are tuned on the first two-thirds of the Alibaba window by time and frozen before the final run; RCAEval cases are never used for tuning. The ablation ladder (critical-path-only → + service tail → full score → MicroRCA) is the centre of RQ2: if critical-path-only already matches the full score, that is the finding. Every comparison is stratified by endpoint so a different request mix in slow traces cannot masquerade as a slow service. Confidence is reported with every ranking and is set to UNAVAILABLE when a window has too few traces or the pipeline is lagging.

What has been run. On October 23 we obtained the RCAEval trace cases (RE2 and RE3 suites for Online Boutique and Train Ticket, 240 cases, 2.6 GB of Parquet traces; the RE1 suite and the Sock Shop cases have no traces). Our reader opened the files unchanged (about 392k spans and 24k traces per Online Boutique case, about 1.1M spans per Train Ticket case), and python -m experiments.run scored the three naive baselines on every case in about 45 minutes on two cores. Table VII and Fig. 18 are the result. The naive heuristics are weak, which is what the project needs them to be: the best of them finds the injected service first in 36% of cases and in the top three in 63%. Two observations already shape the next step. First, critical-path-only ranking beats slowest-average on accuracy@3 and MRR but not on accuracy@1, because in a single-root shop the front end is on every critical path and absorbs the top rank; the full score's deviation-from-baseline signal exists to remove exactly that effect. Second, every method scores zero on the RE3 Online Boutique cases: 12 of those 30 cases inject into adservice or cartservice, which have no spans at all in the recorded traces, and the other 18 inject code-level faults into emailservice and currencyservice that change errors rather than timing. On the Online Boutique cases the critical-path heuristic puts the front end first in 114 of 120 cases. Trace-only ranking

cannot localize a service that is not instrumented, and timing-only heuristics cannot see error faults, so the error-rate signal and an explicit "not observable" output are part of the score design rather than optional.

Method	Cases	acc@1	acc@3	MRR	RE2-OB acc@1	RE2-TT acc@1	RE3-OB acc@1	RE3-TT acc@1
slowest_average	240	0.24	0.34	0.39	0.36	0.18	0.00	0.33
longest_span	240	0.36	0.63	0.51	0.08	0.68	0.00	0.60
critical_path_only	240	0.24	0.55	0.45	0.07	0.40	0.00	0.50

TABLE VII

NAIVE BASELINES ON THE REAL RCAEVAL TRACE CASES (90 RE2-OB, 90 RE2-TT, 30 RE3-OB, 30 RE3-TT). PRODUCED BY EXPERIMENTS/RUN.PY; PER-CASE RANKS ARE IN RCAEVAL_RESULTS.CSV

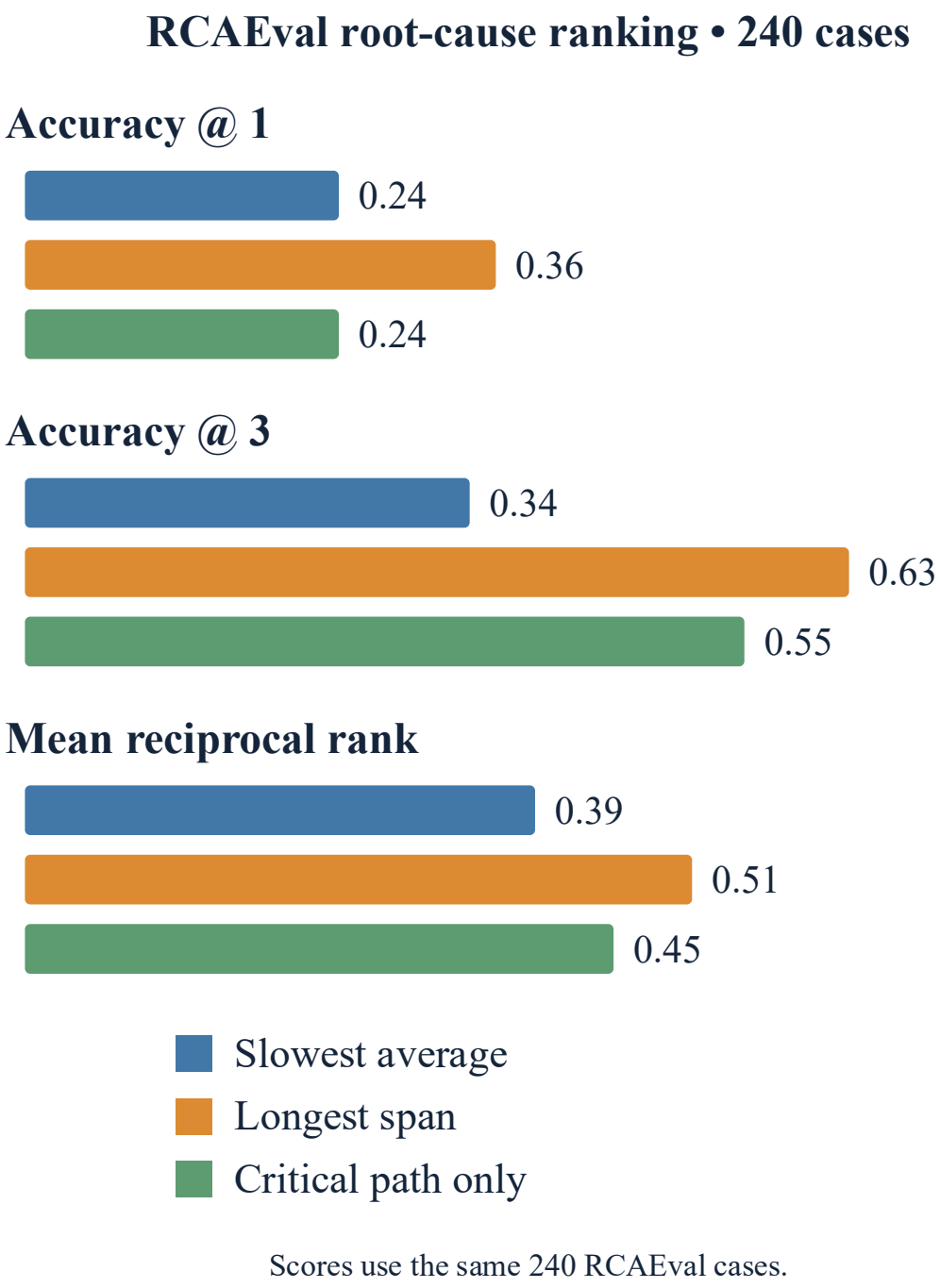


Fig. 18. Baseline ranking performance on 240 RCAEval cases, comparing accuracy at ranks 1 and 3 and mean reciprocal rank for slowest-average, longest-span, and critical-path-only methods.

Human comparison. The usability study (usability/protocol.md) compares manual Jaeger against the dashboard on matched episode sets with counterbalanced groups; it measures time to first plausible diagnosis, time to correct diagnosis, top-1/top-3 agreement and traces inspected. With six to twelve classmates it is exploratory evidence and will be reported as such.

Performance validation plan

Requirement	How we measure	Current measurement	Target / plan
N1 freshness	sta.received_us and sta.normalized_us stamps on every span; /api/pipeline reports p50/p95/p99 staleness; runner's freshness step pulls it after each replay	local pipeline on demo data: p95 under 2 s (no network, no queue)	p95 < 60 s at 1x on AWS, 30–60 min segments; Sprint 6
N2 throughput	replay the Alibaba sample at 10x with HttpSink; compare spans sent vs spans stored	300k-row Alibaba file converts in 6–10 s locally (30–50k rows/s depending on the machine); replay at max speed not yet run against AWS	zero dropped spans at 10x; Lambda concurrency allowed to scale; Sprint 6
N3 cost	cost allocation tags per component; AWS Budgets at $10/$25/$50; experiments/cost.csv filled from the bill	September $0 (everything local); October bill pending	$1–2 idle month, $10–30 active month, $30–60 semester
N4 correctness	replay the same batch twice; count rows	reconstruction tests + adapter tests show identical output on re-run	same check on AWS after deployment
Dashboard latency	timed API calls on demo data (74 traces, 600 spans)	/api/services 4 ms, /api/traces 6 ms, /api/compare 40 ms, /api/flame 25 ms (JSONL store); trace_v1 store adds one round trip per query	p95 < 500 ms on 10k traces; precomputed features from the diagnosis step when ready

TABLE VIII

PERFORMANCE VALIDATION PLAN AND CURRENT NUMBERS

Deployment plan

Contributed by: Matthew Tang (drafted; AWS steps to be confirmed by Matthew)

Environments. Local: compose.yaml starts PostgreSQL 16 on port 55432 with the schema applied, the dashboard runs with uvicorn, and the replayer writes canonical JSONL or posts OTLP to a local collector stub. AWS dev: one CDK stack in us-west-2 named sta-dev, torn down after experiments. AWS demo: the same stack kept up from the end of November for the December 4 demonstration. Promotion is a redeploy of the same stack with a different name, so there is no configuration drift between environments.

Resource	Settings	Owner
S3 raw bucket	uploads/ and raw/ prefixes, 30-day lifecycle, CORS for presigned PUT, event notification → SQS	Matthew
SQS queue + DLQ	visibility timeout 6× adapter timeout, maxReceiveCount 3, alarm on DLQ depth > 0	Matthew
Collector Lambda	Function URL, auth NONE + bearer token check in code, arm64, 256 MB, 10 s timeout	Matthew
Dataset Adapter Lambda	SQS trigger, batch 10, partial batch responses, arm64, 512 MB, 60 s; code from ingest/traceingest/adapter_lambda.py	Anukrithi (code), Matthew (deploy)
S3 normalized bucket	normalized/ prefix, event → reconstruction	Matthew

Resource	Settings	Owner
Reconstruction Lambda	S3 trigger plus 1-minute schedule for quiet-window polls; python -m trace_reconstruction	Larry (code), Matthew (deploy)
Diagnosis Lambda	schedule after settlement; writes trace_analyses and service_window_results	Larry
Aurora Serverless v2	PostgreSQL 16, 0.5–2 ACU, auto-pause after 10 min idle, schema from trace_schema/schema.sql, credentials in Secrets Manager	Matthew
Dashboard API Lambda	Function URL, FastAPI + Mangum (dashboard/app/lambda_handler.py), STORE=trace_v1, 512 MB, 30 s	Anukrithi (code), Matthew (deploy)
CloudWatch + Budgets	7-day log retention, alarms on Lambda errors, DLQ depth, Aurora ACU; budgets $10/$25/$50 emailing all three	Matthew

TABLE IX

DEPLOYMENT RESOURCES

Rollout. (1) Deploy the stack with cdk deploy; outputs are the two Function URLs and bucket names. (2) Run trace_schema/check_schema.py against Aurora. (3) Upload the 10-trace sample through the dashboard and confirm it appears (smoke). (4) Replay the Alibaba sample at max speed and compare counts (N2, N4). (5) Enable alarms. Rollback is cdk deploy of the previous commit; data is never migrated in place because result tables are append-only. Teardown is cdk destroy; buckets with data are retained by policy so a destroy does not lose a dataset.

Reproducing the current implementation

The steps below rebuild everything that exists today on one machine with Python 3.11+ and Docker. They are the same steps the test suites use.

```
git clone https://github.com/LarryTNguyen/MastersProject && cd MastersProject
docker compose up -d postgres                    # PostgreSQL 16 on :55432, schema trace  v1 applied
pip install -r reconstruction/requirements.txt -r dashboard/requirements.txt -r experiments/requirements.txt

# 1. convert a dataset into canonical JSONL (auto-detects Alibaba / RCAEval / OTLP)
cd ingest && python -m traceingest convert ../data/alibaba_sample.csv --out ../out/spans.jsonl --run-id 11111111-1111-1111-1111-111111111111

# 2. reconstruct into PostgreSQL
cd ../reconstruction
python -m trace_reconstruction init-run 11111111-1111-1111-1111-111111111111 --identity-kind DATASET
python -m trace  reconstruction ingest ../out/spans.jsonl
python -m trace_reconstruction finish 11111111-1111-1111-1111-111111111111

# 3. dashboard on the real tables
cd ../dashboard
STORE=trace_v1 DATABASE_URL=postgresql://trace:trace@localhost:55432/trace uvicorn app.main:app --port 8000
# open http://127.0.0.1:8000  (or STORE=jsonl DATA  DIR=../out for the file store)

# 4. tests and the experiment runner
(cd ingest && python -m unittest discover -s tests -t .)
(cd reconstruction && python -m unittest discover -s tests -t .)
(cd dashboard && TEST  DATABASE  URL=postgresql://trace:trace@localhost:55432/trace python -m unittest discover -s tests -t .)
python -m experiments.run experiments/config.json   # cases: data/rcaeval/re2*_*, re3*_* from Hugging Face
```

A one-command demo (dashboard/tools/demo.sh) generates demo data with fresh timestamps and starts the dashboard on the JSONL store; docs/demo-script.md walks through the five views in the order of the dashboard section in Chapter 6.

Bill of materials

Item	Purpose	Idle month	Active month
AWS Lambda (5 functions, arm64)	collector, adapter, reconstruction, diagnosis, dashboard API	$0	$0–5
Amazon SQS	upload and normalization queue + DLQ	$0	$0–1
Amazon S3 (2 buckets)	raw and normalized spans, 30-day lifecycle	<$1	$1–3
Aurora Serverless v2 PostgreSQL	trace_v1 schema	~$1 (storage)	$5–15
CloudWatch	logs (7-day), alarms	<$1	$1–3
Jaeger all-in-one (Docker, local)	manual baseline	$0	$0
Datasets: Alibaba 2021, RCAEval RE2, OTel Demo	evaluation	$0	$0
Total		~$1–2	~$10–30 (semester $30–60)

TABLE X

BILL OF MATERIALS. NO HARDWARE IS PURCHASED; THE ALWAYS-ON COMPARISON (SMALL EC2 + JAEGER + ELASTICSEARCH, ~$50–150/MONTH) IS COSTED FROM LIST PRICES FOR RQ4

Chapter 8. Implementation Plan and Progress

Contributed by: Larry Nguyen

Our implementation will follow an incremental approach in which we first establish the development environment and understand the individual technologies, then connect the components into an end-to-end prototype. Our main evaluation data is the Alibaba production trace collection and the labeled RCAEval benchmark, replayed through the pipeline. For a live testing environment, we will use the OpenTelemetry Astronomy Shop, a sample e-commerce microservice application, to run fault-injection smoke tests. The project will result in a serverless distributed-systems bottleneck analyzer that accepts live or replayed OpenTelemetry traces, reconstructs those traces, ranks likely bottleneck services, and displays the results through a web dashboard.

Programming and execution environment:

- A shared repository and local development environment has been established with the tech stack sorted out. The system uses Python for ingestion, reconstruction, analytics, and replay components; a web development environment for the dashboard; PostgreSQL for local storage (Aurora Serverless v2 in the cloud); and AWS for cloud execution.

Development tools:

- Each team member has access to GitHub, Python, Docker, AWS CLI, AWS CDK, PostgreSQL, OpenTelemetry tools, and the frontend runtime. A shared AWS account with billing alarms is being set up this sprint, with development permissions for each member.

Prototyping and simulation environments:

- Docker Compose runs the OpenTelemetry Astronomy Shop and PostgreSQL locally, the trace replayer feeds Alibaba and RCAEval data into the pipeline, and a separate AWS development deployment is used for cloud testing.

Sample programs:

- We selected the OpenTelemetry Astronomy Shop as our sample e-commerce application, with Google's Online Boutique as a fallback. It ships with a load generator that simulates shoppers and built-in failure flags for injecting faults; getting it running locally is this sprint's task. We have already downloaded and parsed a sample of the Alibaba traces (median call time about 1 ms, p99 about 237 ms), which confirmed the slow-tail behavior our ranking targets.

Clear prototype implementation plan:

- The remaining work is divided into streaming, reconstruction, analysis, dashboard, reliability, evaluation, and release milestones. Each phase has a measurable completion checkpoint. The sprint-by-sprint tasks for each member and the checkpoint for each sprint are in Chapter 9, Project Schedule.

Progress as of October 22, 2026
Contributed by: Anukrithi Myadala (progress tracking), with Larry Nguyen and Matthew Tang

Vertical slice achieved. The whole path from a dataset file to a diagnosis screen now runs on one machine: a converted Alibaba or RCAEval file goes through the adapter, the reconstruction worker, PostgreSQL 16 with the team schema, and the dashboard API, and the page shows traces, waterfalls, slow-versus-normal evidence and pipeline freshness. This is the vertical slice the plan asked for before the Essential features are finished. The live OTLP path and the cloud deployment are the parts of the slice that are still local-only.

Component	Owner	Plan (Workbook 1)	State on Oct 22	Evidence in the repository
Programming environment	all	Sprint 1–2	done	compose.yaml (PostgreSQL 16), requirements per package, README per package
Dataset adapters (Alibaba, RCAEval, OTLP)	Anukrithi	Sprint 3	done, 1 week late	ingest/traceingest/{alibaba,rcaeval,otlp,formats}.py, 52 tests; 300k rows in 6–10 s
Trace replayer (1x/2x/10x/max, rebase)	Anukrithi	Sprint 3	done	ingest/traceingest/replay.py, sinks.py; tests with FakeClock
Upload path (presigned URL, adapter Lambda)	Anukrithi	Sprint 3	code done; deploy pending	upload.py, adapter_lambda.py, POST /api/uploads
Canonical schema v1 + PostgreSQL schema	Larry	Sprint 2–3	done	trace_schema/canonical-span.schema.json, schema.sql, validate.py, check_*.py
Reconstruction (grouping, dedupe, parents, settlement, revisions)	Larry	Sprint 3	done	reconstruction/trace_reconstruction, 24 tests, check_database.py
Self time, critical path (per trace)	Larry / Anukrithi	Sprint 4	dashboard version done; diagnosis step in progress	dashboard/app/analysis.py (18 tests); trace_analyses.span_features reserved
Bottleneck score, baselines table, window results	Larry	Sprint 4–5	in progress	service_window_results and service_baselines tables exist; scoring code not yet pushed
Dashboard: search, filters, detail, waterfall; API	Anukrithi	Sprint 4	done, ahead	dashboard/app, static/index.html, 76 tests
Evidence, confidence, flame graph, pipeline health (dashboard side)	Anukrithi	Sprint 5	done, ahead	/api/rankings, /api/compare, /api/flame, /api/pipeline and their views
Jaeger baseline + usability protocol	Anukrithi	Sprint 5	done, ahead	baselines/jaeger, experiments/usability/protocol.md

Component	Owner	Plan (Workbook 1)	State on Oct 22	Evidence in the repository
Experiment runner, naive baselines, metrics, figures	Anukrithi	Sprint 6	done; first real results Oct 23	experiments/ (11 tests); 240 RCAEval cases scored, see Chapter 7
Collector Lambda, SQS, S3 raw	Matthew	Sprint 3	S3 bucket and SQS created; collector in progress	AWS account (not in repo yet)
Dashboard API Lambda, Aurora Serverless v2	Matthew	Sprint 4	not started	—
Fault-injection smoke test (Astronomy Shop)	Larry	Sprint 3	fixture and test exist; full run pending	reconstruction/smoke/astronomy_shop.py
MicroRCA baseline	Larry	Sprint 5	not started	plugs into experiments/baselines via extra_methods

TABLE XI

IMPLEMENTATION PROGRESS BY COMPONENT (NEW SINCE WORKBOOK 1)

Overall the project is on schedule with uneven distribution: the data-in side, dashboard and evaluation harness are one to two sprints ahead of the plan, the reconstruction step is on plan, and the cloud deployment is about two weeks behind because of midterms and interviews in the second week of October. The team agreed on October 20 to move Matthew's deployment work to the front of Sprint 4 and to pair Anukrithi with him on the adapter and dashboard Lambdas so the end-to-end run on AWS happens in Sprint 4 instead of Sprint 5.

Repository activity since Workbook 1: 36 commits on main between October 14 and October 21 (34 by Anukrithi across ingest, dashboard, experiments, baselines, docs and meeting notes; Larry's trace schema and reconstruction merged on October 20 as pull request #1). Weekly team notes and advisor notes are kept in meetings/.

Risks carried into Sprint 5: (1) the RCAEval download, blocked on home networks until October 23, is resolved: the 240 trace cases are downloaded and the naive baselines are scored (Chapter 7); the remaining risk is that 12 of the 30 RE3 Online Boutique cases inject into services that do not appear in the traces, so those cases will be reported as not observable rather than as misses. (2) Alibaba records carry durations but no absolute timestamps; the team must decide whether to store the replay clock as time_domain RELATIVE so windowed analysis can include Alibaba data (decision due at the October 23 team meeting). (3) Until the collector and adapter are deployed, freshness can only be measured locally.

Chapter 9. Project Schedule

Contributed by: Larry Nguyen (sprint plan, Workbook 1); updates in bold by Anukrithi Myadala

The sprint plan from Workbook 1 is repeated below with a status column added for Workbook 2. Changes since the October 1 submission are in bold: completed items are marked, two of Anukrithi's Sprint 5–6 items were pulled forward and finished in Sprint 4, and Matthew's Sprint 3 deployment item carries into Sprint 4. Figures 19 and 20 are the Gantt charts for both semesters and Fig. 21 is the PERT network with the critical path.

Sprint	Anukrithi	Larry	Matthew
Sprint 1 Sep 7 – 18	Research prior implementations for inspiration. Research potential dashboard implementations. Research trace datasets and different sample applications.	Research potential bottleneck-detection formulas. Research AWS services based on costs, storage, reliability. Create architecture diagram.	Research AWS services based on costs, storage, reliability. Create architecture diagram. Set up AWS account.
Sprint 2 Sep 21 – Oct 2	Define dataset normalization requirements. Create the trace replayer skeleton.	Define the canonical span and trace models. Design the database schema. Create trace schema for sample applications.	Configure AWS account with local environments. Build basic collector Lambda prototype.
Sprint 3 Oct 5 – 16	Implement dataset upload trace replay. Normalize dataset records into common span format (Alibaba and RCAEval converters). Add JSON test inputs. **DONE Oct 14 (one week late).**	Implement span grouping, deduplication, parent-child reconstruction, quiet window completion, and unit tests. Store traces in PostgreSQL locally (moves to Aurora in Sprint 4). Implement fault-injection smoke test on the Astronomy Shop. **DONE Oct 20 (merged as PR #1); smoke test fixture only.**	Deploy collector Lambda with a Function URL (live path, tentative), SQS, S3 raw archive, permissions, retries, and dead-letter handling. **S3 and SQS created; collector carried into Sprint 4.**
Sprint 4 Oct 19 – 30	Build dashboard trace search, filtering, trace details, and waterfall visualization. Connect frontend to dashboard API. **DONE Oct 21; also owns the dashboard API (FastAPI + Lambda handler). Pulled forward from Sprint 5: evidence, confidence, flame graph, pipeline freshness views (done). Pulled forward from Sprint 6: experiment runner, naive baselines, metrics, figures (done). Workbook 2 chapters 6, 7 and progress tracking.**	Calculate self time and critical path. Implement bottleneck score and experiment with different scores. **In progress; dashboard-side self time and critical path available to compare against.**	Deploy dashboard API Lambda with a Function URL and set up Aurora Serverless v2. Connect dataset upload trace replay with AWS services. **Plus carried-over collector deployment; Anukrithi pairs on adapter and dashboard Lambdas.**

Sprint	Anukrithi	Larry	Matthew
Sprint 5 Nov 2 – 13	Add bottleneck evidence, confidence, flame-graph visualization, and pipeline-freshness indicators. Set up Jaeger on the same traces for the manual-diagnosis baseline. **Both done early. New for Sprint 5: first end-to-end run on real Alibaba data in AWS; freshness measurement against the 60 s target; wire the diagnosis results into the dashboard once they land; design-spec follow-ups.**	Implement latency and error baselines, tail keep/drop policy, late-span reprocessing, and incomplete-trace handling. Document the first diagnosed case. Implement the MicroRCA baseline.	Add CloudWatch monitoring, alarms, cost tracking, partial-batch handling, recovery tests, and Aurora auto-pause tuning.
Sprint 6 Nov 16 – 27	Create the one-command experiment runner. Execute RCAEval runs, usability tests, and a timed Jaeger-versus-dashboard comparison. Prepare evaluation figures and the demonstration script. **Runner, figures and demo script done early; Sprint 6 is the RCAEval runs on real cases and the usability sessions.**	Complete two or three case studies. Compare the ranking against the naive heuristics and MicroRCA. Tune and freeze the scoring method.	Run throughput and scaling tests. Measure p50, p95, and p99 freshness. Produce the AWS cost comparison and deployment documentation.
Sprint 7 Nov 30 – Dec 4	Debug the dashboard and create a demo to showcase the workflow.	Make sure that results are reproducible.	Maintain AWS deployment and prepare demo environment.

TABLE XII

SPRINT PLAN WITH STATUS (CHANGES SINCE WORKBOOK 1 IN BOLD)

Sprint	Checkpoint	Status
Sprint 1	Documented technology alternatives, related systems, datasets, and initial architecture.	**met**
Sprint 2	Development environments work, contracts are documented, and a sample trace can be processed locally. Workbook 1 submitted (Oct 1).	**met**
Sprint 3	One real trace travels from the replayer through AWS and is stored as a reconstructed trace.	**partly met: the trace travels replayer → adapter → reconstruction → PostgreSQL locally; the AWS hop is Sprint 4**
Sprint 4	A user can upload or replay traces and view a ranked bottleneck in the dashboard. Workbook 2 and design specifications submitted (Oct 29).	**in progress: upload, replay and dashboard work; ranking waits on the scoring step**
Sprint 5	The system handles duplicate, late, incomplete, and failed records and explains its diagnosis.	**duplicate/late/incomplete handling already in reconstruction; explanation views built**

Sprint	Checkpoint	Status
Sprint 6	Evaluation is complete, major defects are resolved, and no new features are accepted.	
Sprint 7	Working prototype, polished UI, reproducible deployment, completed evaluation, and rehearsed demonstration. Demo, report chapters 1–2, and contribution forms submitted (Dec 4–5).	
Spring 2027 (CMPE 295B)	Final measurement campaigns (Jan–Feb), results frozen, suggested-fix layer, full project report, and Project Expo demonstration (May).	

TABLE XIII

SPRINT CHECKPOINTS WITH STATUS (STATUS COLUMN NEW)

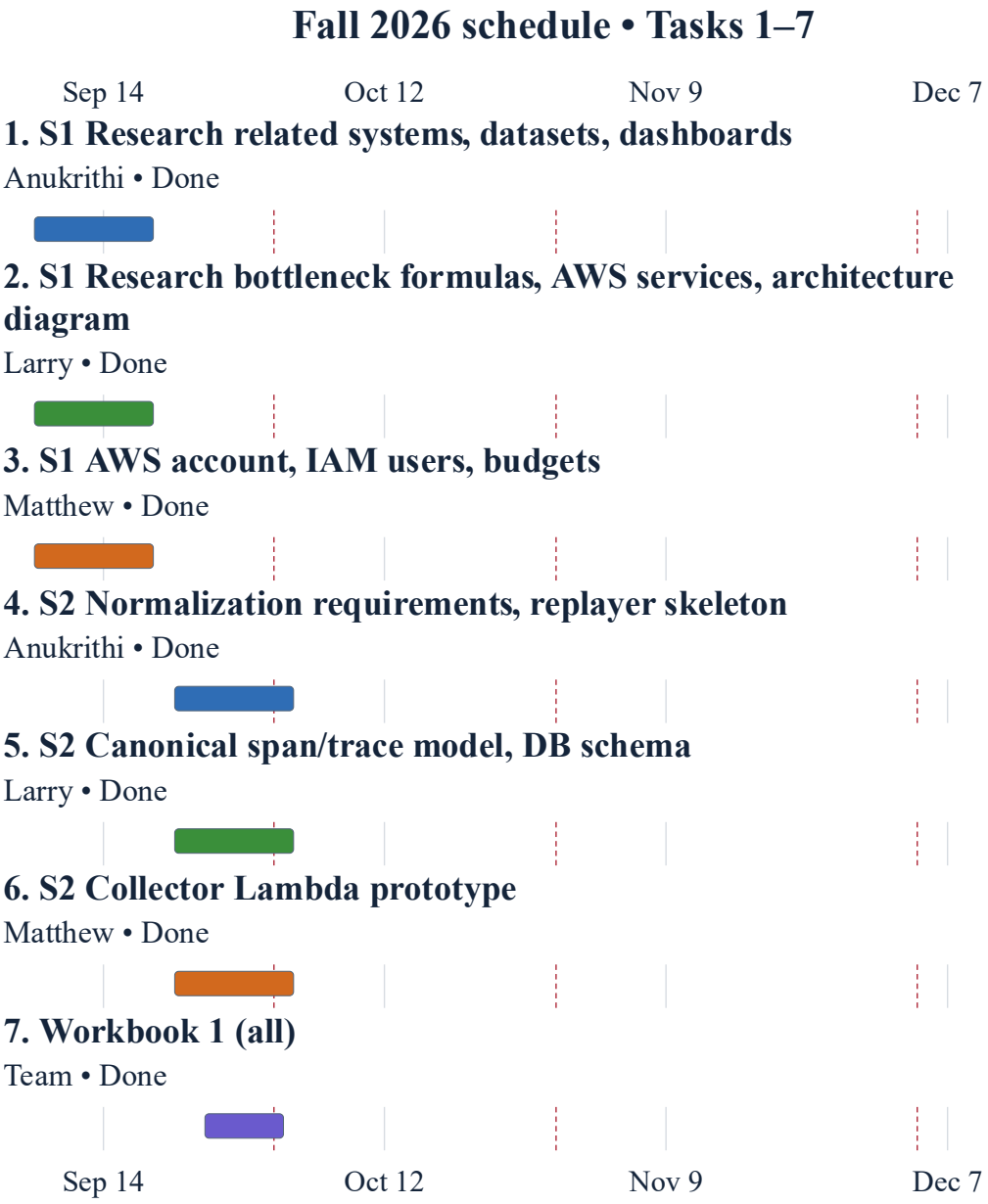


Fig. 19(a). Fall 2026 project schedule, tasks 1–7. Bar positions, owners, and status are retained from the original chart.

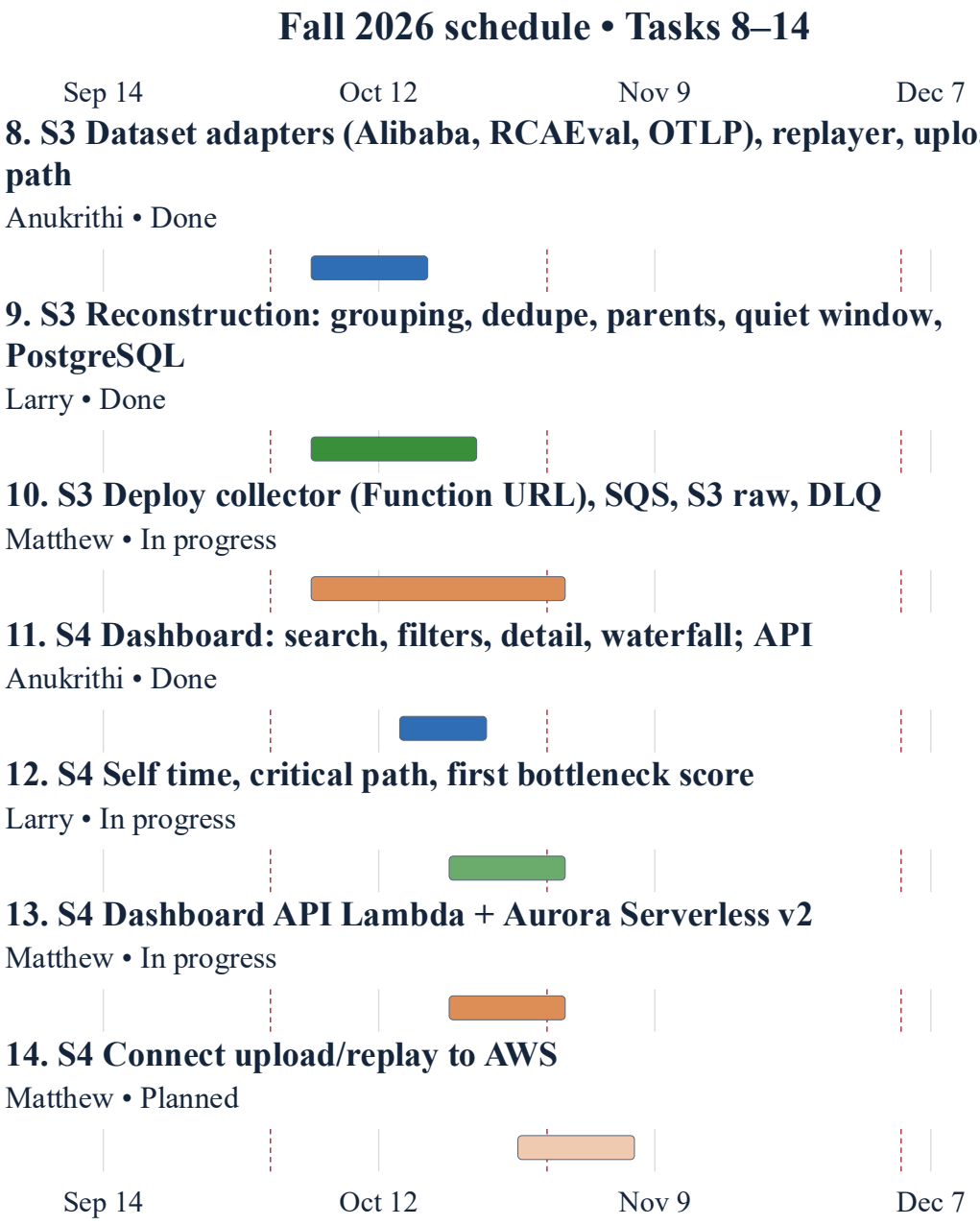


Fig. 19(b). Fall 2026 project schedule, tasks 8–14. Bar positions, owners, and status are retained from the original chart.

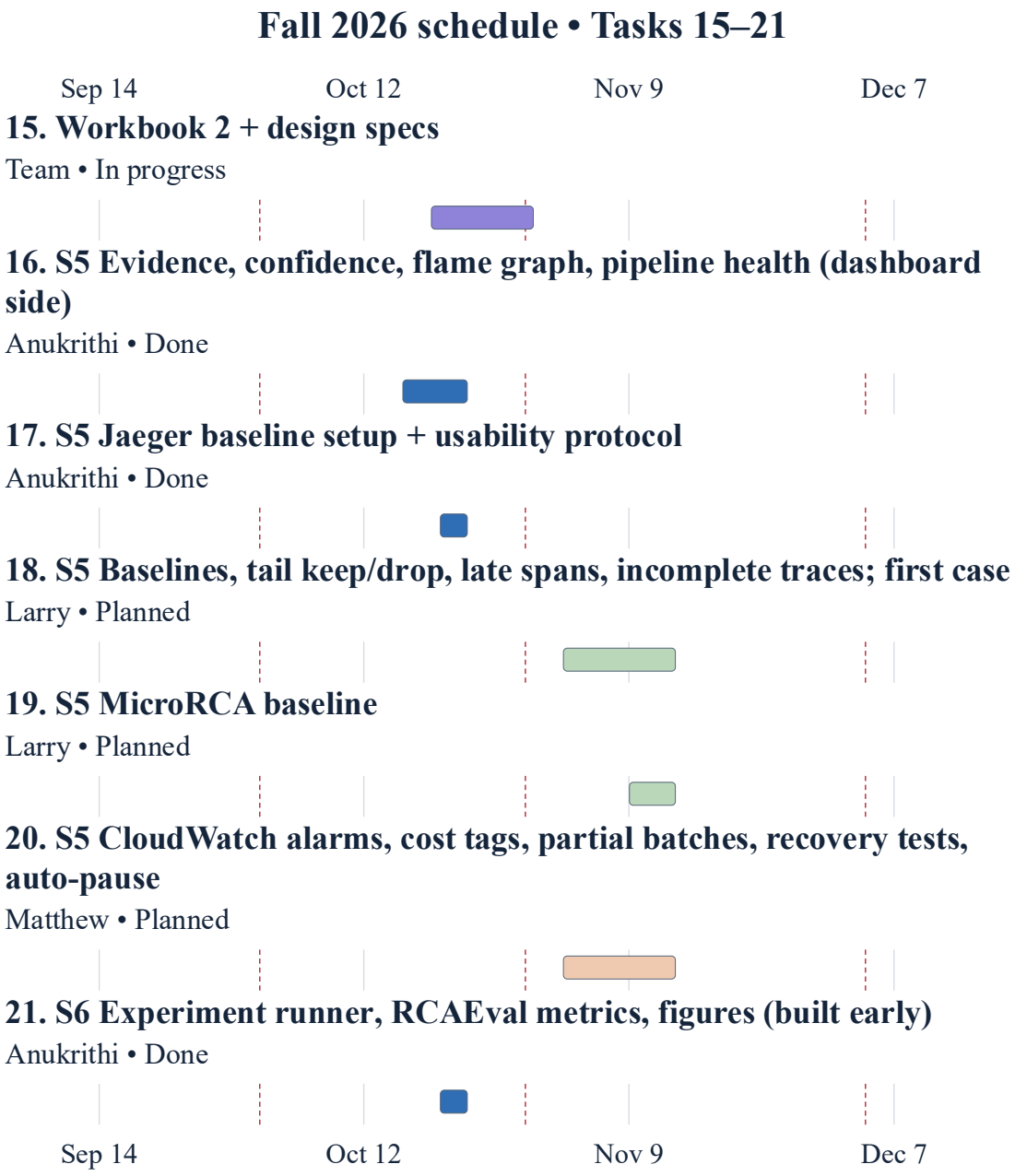


Fig. 19(c). Fall 2026 project schedule, tasks 15–21. Bar positions, owners, and status are retained from the original chart.

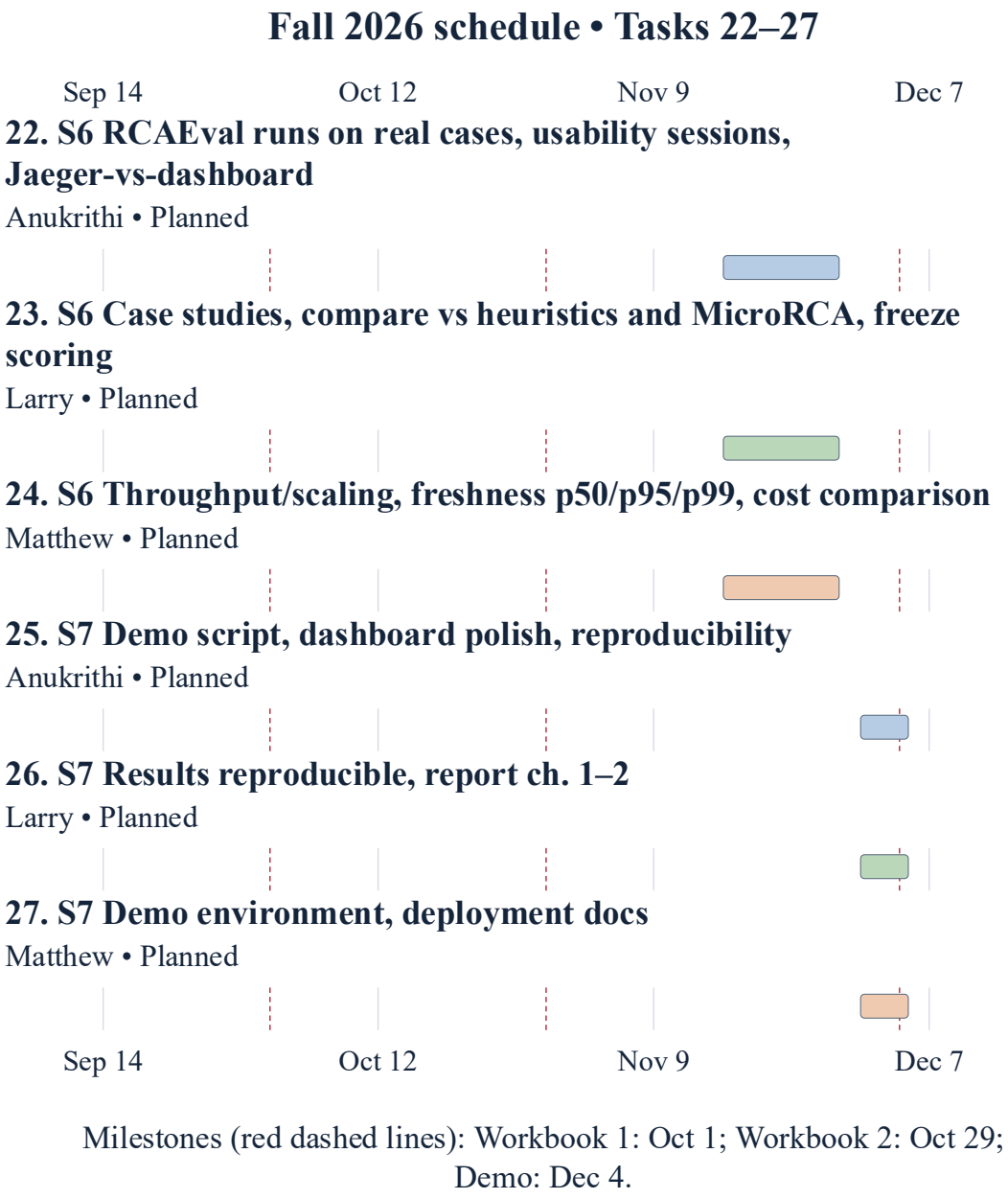


Fig. 19(d). Fall 2026 project schedule, tasks 22–27. Bar positions, owners, and status are retained from the original chart.

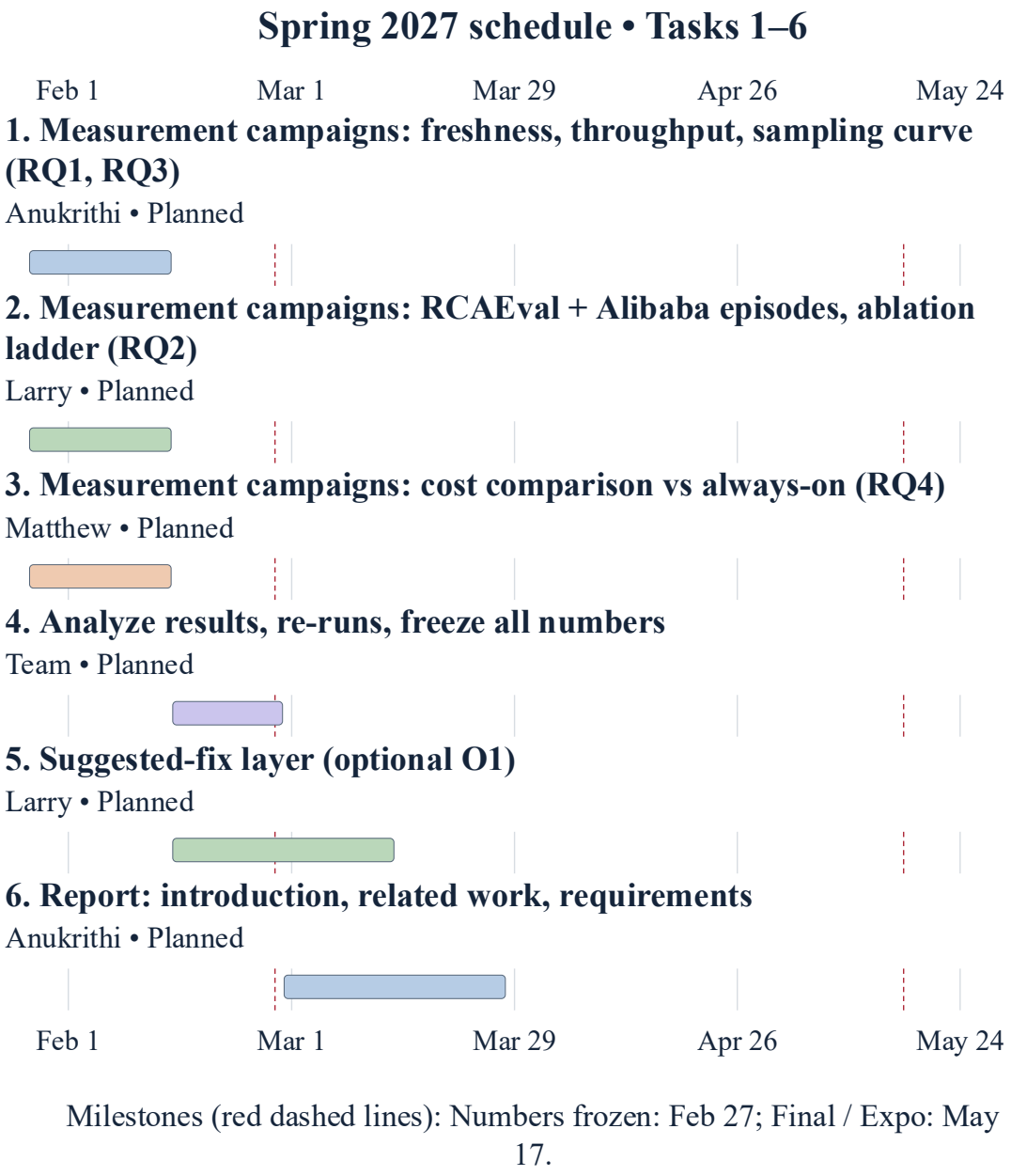


Fig. 20(a). Spring 2027 project schedule, tasks 1–6. Bar positions, owners, and status are retained from the original chart.

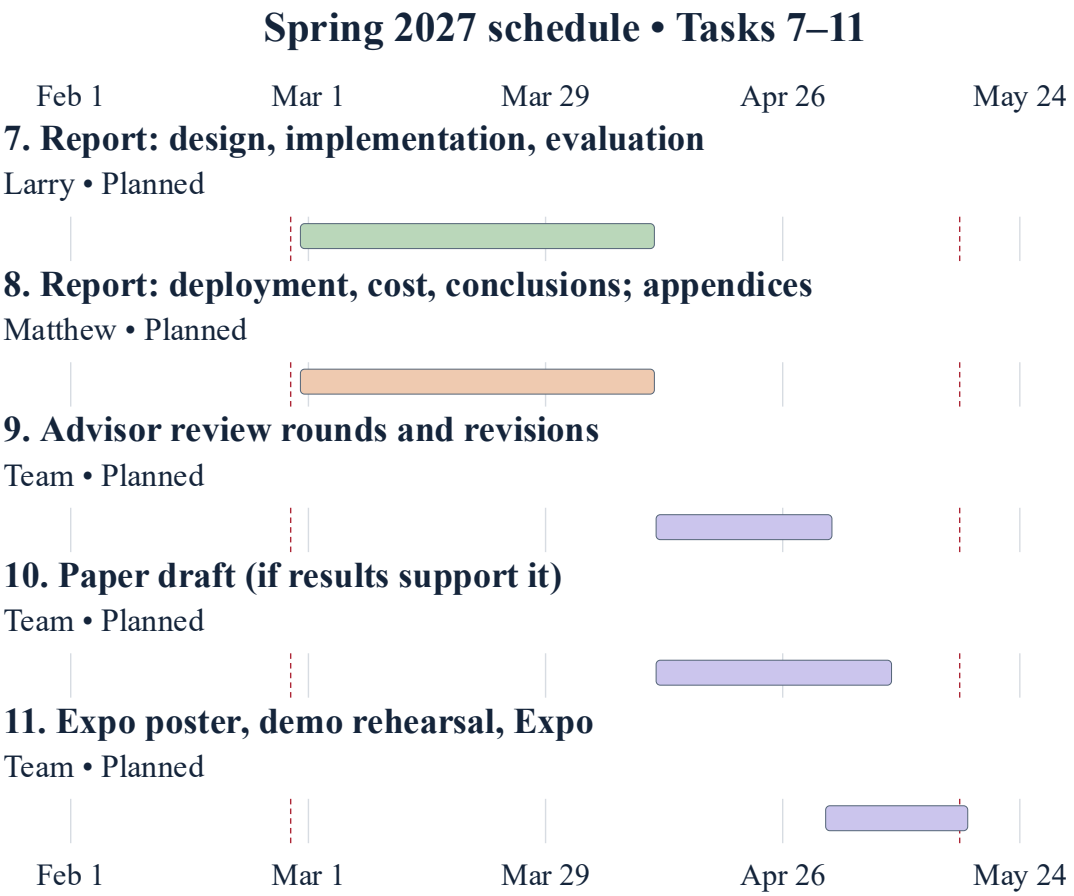


Fig. 20(b). Spring 2027 project schedule, tasks 7–11. Bar positions, owners, and status are retained from the original chart.

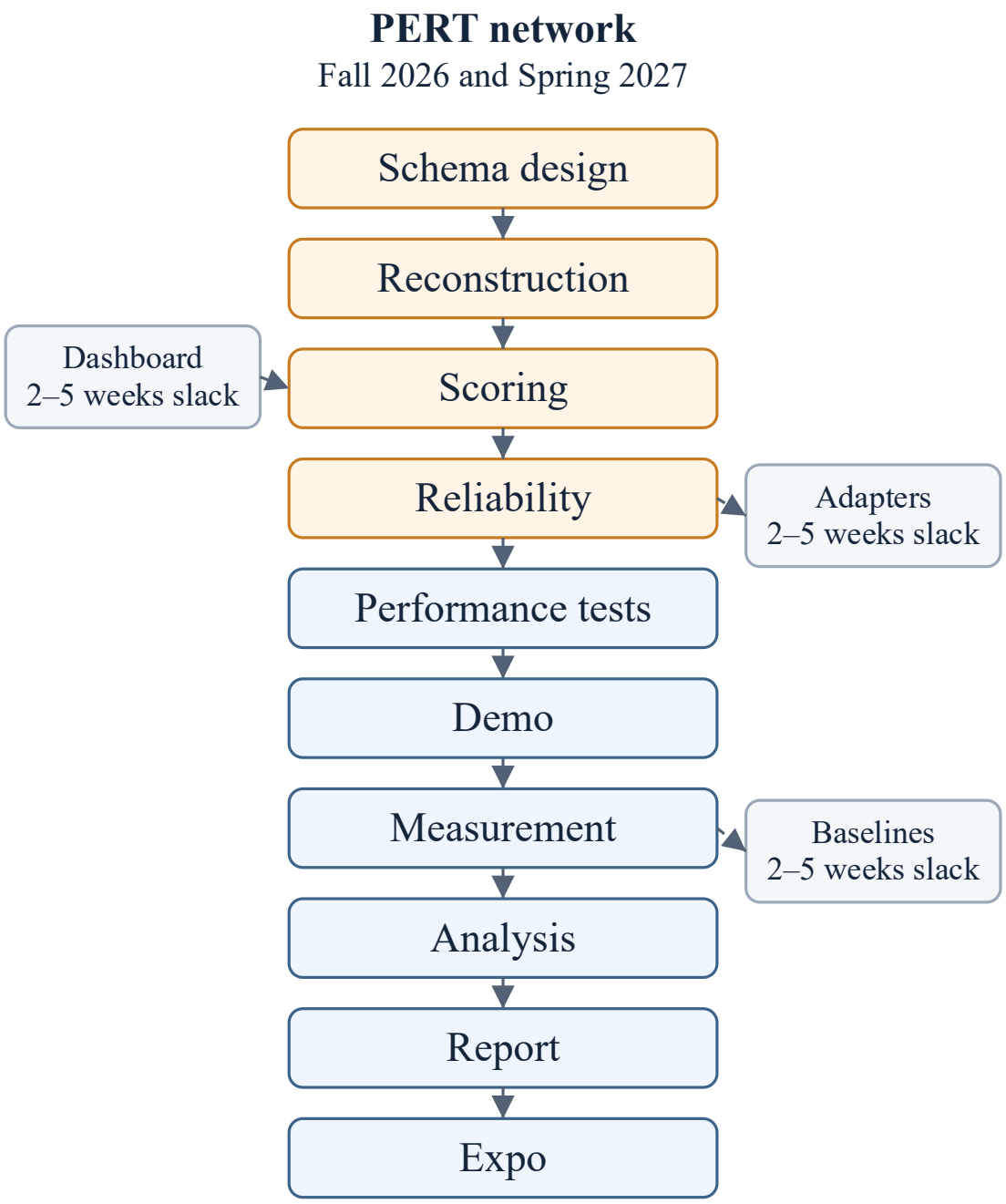


Fig. 21. Project dependency overview for Fall 2026 and Spring 2027, from schema design through reconstruction, scoring, reliability, evaluation, reporting, and Expo. Side branches show dashboard, adapter, and baseline tasks with 2–5 weeks of schedule slack.

Spring 2027 plan (unchanged in dates, now with owners). Jan 27 – Feb 13: measurement campaigns with the runner built in Fall: freshness, throughput and the sampling curve

(Anukrithi), RCAEval and Alibaba episodes with the ablation ladder (Larry), cost comparison against the always-on stack (Matthew). Feb 14 – 27: analyze, re-run anything that looks off, freeze all numbers; optional suggested-fix layer (Larry). Feb 28 – May 17: the full project report chapter by chapter with advisor review rounds, the paper draft if the results support it, and the Expo poster and demo. The 295B deadlines will be aligned to the syllabus in the first week of the semester.